Vehicular Communications 62 (2026) 101073

Contents lists available at ScienceDirect

Vehicular Communications

journal homepage: www.elsevier.com/locate/vehcom

Context-Aware forwarding unavailability prediction in the internet of vehicles: A blockchain-assisted approach

Huda Abualola, Shakti Singh, Rabeb Mizouni *, Hadi Otrok

Computer Science Department, Khalifa University, Abu Dhabi, United Arab Emirates

ARTICLE INFO

Keywords:
Internet of vehicles
Behavior
Context-Aware
Forwarding
Crowdsourcing
Ad-hoc networks
Prediction
Machine learning
Routing
Blockchain

ABSTRACT

This paper tackles forwarding unavailability prediction in crowdsourced Internet of Vehicles (IoV), where rational and independently operated vehicles contribute their resources to support packet forwarding and multi-hop connectivity. Although incentives can encourage participation, user-owned vehicles may become unavailable during their trips due to contextual factors related to trip conditions, resources, environment, or user preferences, causing packet drops, route breaks, higher delay, and reduced packet delivery. Existing works mainly focus on malicious forwarding behavior or network-level prediction, while overlooking context-driven forwarding unavailability caused by rational vehicle decisions. Moreover, blockchain-enabled solutions support transparent dissemination of behavior-related information, but usually keep the prediction process off-chain, limiting its verification in decentralized environments. To address these limitations, this paper proposes a blockchain-assisted context-aware framework that predicts a vehicle's forwarding unavailability probability before route establishment in crowdsourced IoV using temporal, spatial, and weather-related context. Each vehicle uses a lightweight XGBoost model trained on its trip behavior data and deployed on-chain through a Forwarding Behavior Predictor smart contract, while the predicted probability and validation parameters are recorded and disseminated via a Prediction Registry smart contract. The prediction is then integrated into Forwarding Behavior-Aware QoS-OLSR to proactively avoid vehicles likely to become unavailable during relay selection and routing-cost computation. Simulation results show improved packet delivery, higher successful flow completion, and fewer router drops compared with the baseline under different contexts and prediction uncertainty.

1. Introduction

The Internet of Vehicles (IoV) is a connected vehicular networking paradigm that enables vehicles to communicate with other vehicles, roadside infrastructure, pedestrians, edge servers, and cloud platforms. Through these communications, IoV supports services such as road safety, traffic efficiency, data dissemination, packet forwarding, and edge-assisted applications [1]. In decentralized and crowdsourced IoV environments, vehicles are not only consumers of network services but also relays that forward packets and help maintain multi-hop connectivity. The continuity of IoV services depends strongly on the availability of participating vehicles to remain engaged in forwarding throughout their trips. Such that, if one or more relay vehicles become unavailable during an active route, it may experience packet drops, route breaks, repeated rediscovery, higher delay, and reduced packet delivery. Therefore, to encourage the voluntary participation of crowdsourced vehicles, crypto-based monetary incentives managed and verified through blockchain-based solutions have been proposed [2,3]. However, even with such incentives, crowdsourced vehicles remain rational and independently operated nodes whose availability cannot be assumed to be stable or guaranteed.

Crowdsourced vehicles are user-owned nodes that contribute their own resources for forwarding services; hence, their forwarding availability is susceptible to contextual factors that affect users' rational decisions [4,5]. These factors may be related to the trip, vehicle resources, environment, or user preferences, including trip origin and destination, travel time, day type, weather conditions, resource limitations, and user preferences [6,7]. For example, a vehicle may become unavailable for forwarding if the user prioritizes available bandwidth for personal applications on a particular day, or if the trip is interrupted, delayed, or canceled due to rainy weather conditions. Such context-driven changes in participation can disrupt established routes, increase packet drops, and degrade routing performance. This motivates the need to proactively predict forwarding unavailability before route

* Corresponding author.
E-mail addresses: huda.abualola@ku.ac.ae (H. Abualola), shakti.singh@ku.ac.ae (S. Singh), rabeb.mizouni@ku.ac.ae (R. Mizouni), hadi.otrok@ku.ac.ae (H. Otrok).

https://doi.org/10.1016/j.vehcom.2026.101073
Received 10 June 2026; Received in revised form 3 August 2026; Accepted 23 August 2026
Available online 24 August 2026
2214-2096/© 2026 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

establishment, rather than reacting to packet drops after routing degradation has already occurred.

Several works in the literature have addressed relay behavior in vehicular and ad hoc networks through mechanisms for detecting, evaluating, and predicting node behavior [8–11]. These solutions mainly focus on identifying malicious packet-forwarding behaviors, including black-hole, gray-hole, and wormhole attacks, as well as intrusive nodes [8,9,12]. This is typically achieved through monitoring, packet-forwarding observations, message inspection, behavior scoring, or prediction models [13]. Other predictive works in ad hoc networks improve routing and relay selection through mobility, link-stability, congestion, or trajectory prediction [14–16]. Although these works improve routing reliability and security, they mainly depend on network-observable indicators and historical communication behavior. They interpret dropping or non-forwarding behavior as malicious and untrusted, while overlooking the rationality of nodes whose availability for routing and forwarding may change due to contextual reasons.

Forwarding unavailability prediction can improve route establishment in crowdsourced IoV by helping vehicles select relays that are more likely to remain available during packet forwarding. However, in a decentralized setting where vehicles are independently operated and no trusted central authority exists, prediction results need to be generated and shared in a transparent and verifiable manner. Blockchain can support this requirement by recording the prediction-related information and outcomes in a decentralized and verifiable manner, allowing participating vehicles to validate the prediction process before using its output in routing decisions. Several blockchain-enabled works in IoV and ad hoc networks use blockchain for behavior record storage, reliability information dissemination, incentive management, or coordination support [11,17,18]. However, in these works, the prediction process remains external to the blockchain, which limits the ability of participating vehicles to verify how prediction outcomes are generated in a decentralized environment.

In this paper, we propose a blockchain-assisted context-aware unavailability prediction framework for crowdsourced IoV, where forwarding participation is treated as a context-dependent behavior of rational vehicles. The term context-aware in this work is used from the perspective of crowdsourced service participation rather than from the conventional physical-network perspective. Specifically, the temporal, spatial, and weather-related features represent conditions that may affect a rational user-owned vehicle's willingness or ability to remain available for forwarding, rather than instantaneous network-state indicators such as channel quality, bandwidth usage, signal strength, or link stability. The framework aims to predict, before route establishment, whether a vehicle is unavailable for forwarding during its upcoming trip using temporal, spatial, and weather-related contextual information. Each participating vehicle is associated with a lightweight XGBoost prediction model that is trained on its own trip-related behavior data. The trained model is deployed on-chain through a Forwarding Behavior Predictor smart contract, which receives the input features of the upcoming trip and generates the forwarding unavailability probability of the vehicle. Since crowdsourced vehicles are independently operated, the proposed framework avoids relying on unverifiable self-reported availability values by executing the lightweight XGBoost predictor on-chain and recording the prediction output for transparent validation. The generated prediction is then recorded and disseminated through a Prediction Registry smart contract together with the associated prediction parameters required for validation, including the vehicle node ID, predictor contract address, selected feature indices, scaling factor, normalization means, and normalization scales. These parameters allow neighboring vehicles to reconstruct the required input vector, execute the corresponding predictor contract, and compare the generated output with the recorded prediction. In this way, neighboring vehicles in the IoV can retrieve the announced prediction before route establishment and use it as part of their routing decisions, while the network can also validate the transparency and correctness of the prediction process. Finally, the predicted forwarding unavailability probability is integrated into the QoS-OLSR routing protocol to propose Forwarding Behavior-Aware QoS-OLSR, where vehicles predicted to become unavailable are proactively avoided by reducing their willingness and QoS role in relay selection and by increasing the routing cost of the links connected to them. This allows the routing protocol to reduce its dependence on vehicles that are likely to stop forwarding due to contextual reasons before such behavior appears as packet drops or route degradation.

In summary, the main contribution of this paper is a framework for crowdsourced IoV that:

- predicts the forwarding unavailability of rational vehicles before route establishment using contextual trip-related and environmental-related features,
- deploys on-chain lightweight vehicle-specific prediction models to execute the prediction in a decentralized and transparent manner,
- disseminates the predicted forwarding unavailability together with the associated prediction parameters required for validation through blockchain smart contracts, and
- integrates the predicted forwarding unavailability into QoS-OLSR routing protocol to develop a forwarding behavior-aware QoS-OLSR, which proactively avoids vehicles that are likely to become unavailable during their trips and improves the routing performance in crowdsourced IoV.

The proposed framework is evaluated through both prediction-model assessment and network-level simulation. The lightweight XGBoost model is compared against baseline prediction models, such as logistic regression, to evaluate its ability to estimate forwarding unavailability while remaining suitable for on-chain execution. The routing impact of the prediction output is then evaluated through NS-3 simulations by integrating the predicted forwarding unavailability probability into QoS-OLSR and comparing it with the baseline QoS-OLSR protocol. The simulations consider different vehicle densities, weather-related contexts, and prediction uncertainty to examine how proactive relay avoidance affects routing performance. The results show that prediction-aware routing improves packet delivery and successful flow completion, reduces router drops, and limits the use of vehicles that are likely to become unavailable, although this can increase hop count and delay when the protocol selects longer alternative paths to avoid unreliable relays.

2. Related work

In this section, the works proposed in the literature on forwarding and routing behavior detection and prediction and the use of blockchain for behavior and trust management in ad hoc networks are discussed.

Forwarding behavior detection has been tackled as malicious black-hole and gray-hole behavior in several works. In [9], the authors tackle the behavior of malicious VANET nodes, particularly black-hole vehicles that drop OLSR control messages. Their solution is to detect malicious behavior using fuzzy logic that computes a trust value from the exchanged control messages, and nodes with low trust are detected and isolated as malicious. Blockchain is used to validate claims, elect validators through Proof-of-Trust, and disseminate the detected attackers' information. Also, dropping behavior, such as black-hole attacks, is also discussed in [8]. The work in [8] proposes a methodology that combines authentication, trust-based route selection, and deep packet inspection using a Bi-Directional GRU to detect and block intrusive behavior. Blockchain is used as a secure decentralized verification and trusted data management. In [19], the authors consider selfish and rational nodes in VANET, where the targeted behavior includes spoofing, jamming, message forgery, and eavesdropping attacks performed for profit. They combine an indirect reputation mechanism with a deep reinforcement learning algorithm to avoid selfish and malicious nodes and motivate cooperation. Blockchain is mainly used to securely record nodes' behaviors and reputation information. The work in [20] addresses malicious and untrusted vehicle behaviors that can send excessive queries,

perform Sybil attacks, or report false location information. Such attacks affect the location privacy preservation in VANETs. The authors propose an improved Bayesian trust model with direct and second-hand evaluations to assess vehicle trustworthiness before constructing anonymous cloaking regions. A lightweight consortium blockchain is used to securely store and validate historical trust information and anonymous-region transactions through RSUs. The work in [21] considers malicious wormhole nodes in wireless ad hoc networks. The authors focus on the post-wormhole actions they perform after creating the wormhole link, such as packet dropping and replaying. The proposed solution depends on local monitoring and isolation based on alerts disseminated by legitimate nodes that observe neighbors' packet behavior. Also, wormhole attacks in MANET are discussed in [22]. The authors propose a reinforcement learning-based solution that uses routing-related metrics to evaluate and compute the trust level of neighbor nodes. Based on a predetermined threshold for trust level, the network nodes decide whether to exclude their neighbors. The work in [23] considers misbehaving vehicles in V2X that send faulty or malicious safety messages, including 19 different misbehavior types such as false position, false speed, disruptive messages, replay, DoS, and Sybil-based attacks. The misbehaving vehicles are detected using a semi-supervised federated learning-based system. The work utilizes edge servers that collaboratively train a multi-class classifier using limited labeled data and large unlabeled message datasets. The authors in [24] also tackle misbehavior in terms of faulty messages. To detect malicious messages, ML-based message classification is refined using a trust score calculated from surrounding vehicles and environmental observations through a modified Logistic Trust model. Another work that focuses on black-hole attacks is proposed in [12]. The authors propose trust-based routing through TSDR and TCOR, where packet forwarding observations and neighbor recommendations are used to detect malicious nodes and maintain trusted routes. In [25], the authors focus on the malicious activities of parked computing vehicles in IoV, such as providing poor-quality service or false information to gain incentives. The malicious behavior detection methodology combines trust evaluation with contract-based incentives, where trustworthy vehicles are selected using direct and indirect trust before contracts are assigned. A blockchain-based solution is proposed in [26] along with a federated learning process, where the blockchain is dedicated to storing trained models. The work focuses on classifying and detecting intrusion and attack behavior in VANETs. Additionally, routing behavior has been tackled in [27] in FANETs, where the authors proposed an SDN-based optical switching integrated with congestion-aware routing, RF fallback, and lightweight trust-aware security. The behavior of nodes is assessed using trust values, such that unreliable UAVs are excluded from routing decisions. The work in [28] also considers malicious and intrusive MANET node behavior, such as packet interception, route manipulation, and unauthorized access. The malicious behavior is detected through a multi-path routing mechanism that utilizes a trust metric, while a Deep Maxout Network is used to classify the intrusion. Another work proposed in [29] targets the dropping behavior, in addition to spoofing and denial of service attacks. The work proposes a detection and classification mechanism that combines sensor-based feature extraction, where features such as confidence level, support level, RSSI, and Time of Travel are extracted, with a Precise Probability Genetic Algorithm for feature optimization, followed by a Stacked Recurrent LSTM classifier to label traffic as normal or attacking. Other related security mechanisms focus on verifying node legitimacy. For example, D2VAuth [30] combines lightweight cryptography and permissioned blockchain consensus to provide secure and resilient authentication in drone-assisted IoV networks. While the framework verifies node legitimacy and protects against several communication attacks, it does not assess whether an authenticated vehicle may rationally become unavailable because of trip-related, temporal, spatial, or environmental conditions. Service disruption due to rational behavior is also tackled in [31], where a coalition and reputation-based energy auction prioritizes reliable EV customers and uses deposits to mitigate no-show behavior in mobile charging services.

The prediction of forwarding behavior has been proposed in the literature as a proactive approach to avoiding malicious and untrustworthy nodes. In this context, AI enables vehicular systems to learn from sensory, traffic, environmental, and behavioral data, thereby supporting the prediction of changing conditions and context-aware decision-making in dynamic IoV environments [32]. In [33], the work tackles malicious vehicles in IoV that perform false messages, bad-mouthing, and on-off attacks. A trust evaluation mechanism is proposed based on a filtering technique, where historical trust is used to predict the current trust state, and this prediction is then fused with current trust observations using a Kalman filter-based approach. Prediction of trust to evaluate nodes' behavior is also proposed in [13]. The work focuses on black-hole and gray-hole attacks that affect packet forwarding and routing reliability. The solution involves collecting a comprehensive set of behavioral features–both static, such as node location and transmission range, and dynamic, such as packet forwarding ratios, topology changes, and packet loss rates–forming a rich feature space for training an LSTM model. The prediction model is integrated with LAR, where node trust is predicted from behavioral routing features and then used directly in next-hop selection. A trustworthiness prediction mechanism is proposed in [11]. The work's objective is to calculate and predict the evolution of trust scores. The work utilizes Markov chains to model risk and reputation transitions and combines these with IDS and vulnerability factors in a trust score function. Blockchain is integrated in [11] as a secure and auditable trust-data management. Additionally, the work in [17] tackles the trustworthiness behavior of vehicles in VANET to disseminate information. The vehicles' trustworthiness is predicted using an ensemble learning model combined with a computed reputation model. The reputation model uses message correctness, vehicle age, participation degree, similarity of messages, message freshness, and sender proximity as behavioral features, while the trust classifier uses network behavioral features such as the IP address of source and destination nodes. Blockchain in [17] is used as a tamper-proof storage for predictions and reputation scores. The work in [10] predicts trust in MANET nodes to forward packets, especially under black-hole and gray-hole attacks. The prediction of trust values is based on a dynamic trust prediction model in which the historical trust of a node is derived mainly from its packet forwarding ratios, including control-packet forwarding ratio and data-packet forwarding ratio, and then uses extended fuzzy logic rules together with the node's capability level to predict its future trust value. This predicted trust is then integrated into a Trust-based Source Routing (TSR) protocol, where routes are selected according to both hop count and route trust requirements. Worker behavior prediction in mobile crowdsensing is also examined in [34], where individualized AI models predict task completion during worker selection, while a GAN-based poisoning attack is introduced to expose the vulnerability of these behavioral models to malicious manipulation. An ensemble-based hybrid intrusion detection model was proposed in [35] to identify multiple attack categories in IoV networks using the CIC-IDS2017 and UNSW-NB15 datasets. While the model achieved high attack-detection accuracy, its objective was to distinguish malicious from benign network activity. An edge-intelligent semantic aggregation framework was introduced in [36] for UAV-assisted 6G IoV networks, combining deep reinforcement learning, semantic communication, and blockchain validation to adapt transmission decisions to traffic and network conditions.

Routing level behavior predictions based on link stability and trajectories of network nodes have been tackled in many works. In [15], the authors tackle the congestion and quality of links in UAV-assisted VANETs. They propose an RNN-GAN probabilistic forecasting with chance-constrained MPC to predict link and congestion distributions and then select robust relay nodes. Congestion of network links, in addition to mobility metrics, is also tackled in [37]. The work focuses on the problem of traffic distribution in FANETs. The authors propose

Table 1
Comparison with related trust, malicious-behavior, and routing prediction works.

Ref.	Focus	Malicious	Predictive	Target	Main Indicators	Blockchain Role
[9]	Black-hole detection, fuzzy trust	Y	N	Node	Control messages	Storage; PoT validator election
[8]	Black-hole detection, DPI + trust routing	Y	N	Node	Packet-level behavior	Secure verification/storage
[19]	Malicious node avoidance, DRL	Y	N	Node	Observed behavior	Reputation storage
[20]	Sybil/false-location detection, Bayesian trust	Y	N	Node	Query/location behavior	Consortium trust storage
[30]	Node legitimacy, lightweight crypto auth.	Y	N	Node	Authentication data	Consensus-based auth.
[33]	Malicious/on-off attack detection, Kalman-fused trust	Y	Y	Node	Trust history	–
[13]	Black/gray-hole trust prediction, LSTM + LAR	Y	Y	Node	Forwarding ratios, topology	–
[11]	Trust-score evolution, Markov chains + IDS	Y	Y	Node	IDS, vulnerability factors	Auditable trust storage
[17]	Trustworthiness prediction, ensemble + reputation	Y	Y	Node	Message, reputation features	Tamper-proof prediction storage
[10]	Trust prediction, fuzzy logic + TSR	Y	Y	Node	Forwarding ratios	–
[14]	Link-stability prediction, Bayesian belief + hedonic game	N	Y	Link	Intersection/traffic info	–
[6]	Location prediction, Kalman filter + two-tier routing	N	Y	Link	Vehicle location	–
[37]	Traffic/link prediction, STGCN + MMJP	N	Y	Link	Topology, buffer state	–
[38]	Link-lifetime prediction, LPAR	N	Y	Link	Link lifetime	–
Proposed	**Forwarding unavailability as rational contextual behavior**	**N**	**Y**	**Node**	**Trip-level temporal, spatial, weather**	**On-chain prediction execution + verifiable registry**

a solution model with the Multi-Metric Joint Prediction (MMJP) protocol that uses a Spatio-Temporal Graph Convolutional Network (STGCN) to jointly model traffic dynamics and topology structure and predict future node resource availability, such as buffer state and link-related conditions. Then it uses these predictions for congestion-aware route selection. Mobility behavior of UAVs in FANETs is also tackled in [38]. The work proposes a Link-stability Prediction-based Adaptive Routing (LPAR) protocol. The mobility is predicted to estimate the lifetime of the links and evaluate stability. Hence, the more stable routes are favored. Stability of links due to mobility behavior is also tackled in [14]. The main idea is to estimate whether neighboring vehicles are likely to remain connected when they approach and leave the nearest intersection. The work considers environmental metrics from urban VANET, such as traffic lights, lane directions, and the time to leave the street segment. Each Node in the network predicts its relative stability compared to each neighbor using Bayesian Belief. The predicted stability is utilized in the QoS calculation that feeds into the hedonic game for relay selection. Additionally, the authors in [16] tackle mobility behavior by predicting vehicles' trajectories in VANET. A federated learning framework is proposed, where a target vehicle selects relevant surrounding vehicles through a vehicle relevance-oriented collaborative node selection algorithm. The selected relevant vehicle is then used as a collaborative training node. These vehicles train the trajectory prediction model locally using observed trajectory data and send back model parameters. In [6], the future locations are predicted to enhance the routing in IoV. A proactive routing protocol is proposed to enhance the stability of links among vehicles. The proposed two-tier routing algorithm utilizes the Kalman filter location prediction model. According to the predicted locations within road segments, the routing protocol classifies the nodes to maintain the stability of routing paths. In addition, multi-agent task allocation and MARL-based coordination methods for IoV were comprehensively reviewed in [39], with particular attention to resource allocation, communication overhead, scalability, and security. While these methods support intelligent task assignment among vehicles, they do not explicitly model the context-dependent decision of a rational vehicle to remain available for an assigned service, which is the focus of the present work.

Table 1 summarizes the most relevant related works and identifies their limitations compared with the proposed framework in terms of the modeled behavior, prediction objective, input indicators, and blockchain role. As shown in the table, the existing works can be grouped into three main categories. The first category includes malicious-behavior and trust-management works, such as [8,9,19,20], and [30]. These works mainly detect, isolate, or authenticate malicious and untrusted nodes using network-observable indicators, such as control messages, packet-level behavior, reputation values, query behavior, location claims, or authentication information. Although these works improve routing security and trust management, they mainly interpret non-forwarding or dropping behavior as malicious, selfish, or untrusted behavior. In contrast, the proposed work does not assume that forwarding unavailability is necessarily malicious. Instead, it models forwarding unavailability as a rational and context-dependent service-participation behavior of independently operated crowdsourced vehicles.

The second category includes trust and reliability prediction works, such as [11,13,17,33], and [10]. These works are predictive, but their prediction targets are mainly trustworthiness, malicious behavior, or future trust scores. Their input indicators are typically network-based, including forwarding ratios, topology changes, message features, trust history, IDS outputs, vulnerability factors, and reputation records. Therefore, these works predict whether a node is trustworthy or likely to misbehave based on past network behavior. In contrast, the proposed framework predicts whether a rational vehicle is likely to become unavailable for forwarding before route establishment using trip-level temporal, spatial, and weather-related features. This distinction is important because a vehicle may remain authenticated, non-malicious, and physically reachable, but may still become unavailable for forwarding due to user decisions, trip conditions, weather, or other service-participation factors.

The third category includes link-stability, mobility, and routing prediction works, such as [6,14,37], and [38]. These works predict route or link stability using mobility-related and network-level indicators, such as vehicle location, intersection behavior, topology, buffer state, congestion, or link lifetime. Their objective is to select routes that are more stable from the physical connectivity or mobility perspective. However,

link stability does not necessarily imply forwarding availability. A vehicle may remain within communication range and maintain a stable link, but may still become unavailable as a forwarding participant because the user decides not to contribute resources during the trip. Therefore, the proposed forwarding unavailability prediction is complementary to link-stability prediction, but it addresses a different source of routing disruption.

The comparison also shows that blockchain is mostly used in existing works for storage, authentication, trust-data management, validator election, or tamper-proof dissemination of externally computed values. In contrast, the proposed framework deploys the lightweight vehicle-specific XGBoost predictor itself on-chain and records the prediction output together with the parameters required for validation through the Prediction Registry. This allows other participating vehicles to reproduce and validate the forwarding-unavailability prediction before using it in routing decisions. Therefore, the novelty of the proposed framework lies in combining rational context-driven forwarding unavailability prediction with verifiable on-chain prediction execution and routing-level integration.

In summary, the main limitations of existing works are,

- They mainly address malicious behavior detection, trust evaluation, or trust prediction, rather than predicting the forwarding availability as non-malicious but rational behavior of crowdsourced vehicles.
- They rely mostly on observed network behavior, such as forwarding ratios, routing traces, or message-level features, and do not capture richer temporal, spatial, and environmental trip context, such as weather.
- Detection-based work remains reactive, since the node behavior is evaluated after being observed, even when the result is later used for trust prediction or route selection.
- Prediction-based routing works mainly target the link, predicting mobility, connectivity, or link stability between neighboring nodes, rather than the node itself and its willingness or availability to forward.
- In blockchain-enabled works, blockchain is typically used for secure storage, trust dissemination, or coordination support, rather than for executing the forwarding behavior prediction model itself.

3. System model overview

The blockchain-based forwarding unavailability prediction model is proposed to provide proactive forwarding decisions for rational vehicles in IoVs, where participants are rewarded and penalized based on forwarding contribution. Therefore, early prediction of forwarding unavailability improves the network performance. An overview of the proposed blockchain-based prediction model is presented in Fig. 1. The main components are:

- **Vehicles:** are the IoV network nodes that act as data forwarders and network users. To disseminate connectivity and enable data forwarding over multiple hops, all vehicles in the network run a routing protocol, namely Forwarding Behavior-Aware QoS-OLSR. Also, each vehicle that joins the network has a trip with a start and end location. All joining vehicles are part of the network; however, they are considered rational nodes that are optionally available to forward data packets during their trip. Deploying the prediction model on-chain enables participating vehicles and routing validators to verify and validate the predicted output in a transparent and verifiable manner, while the predicted probability is logged on-chain for direct real-time use by participating vehicles.
- **Forwarding Behavior Prediction (FBP) Smart Contracts:** are the immutable contracts that execute the forwarding unavailability prediction on-chain. Each vehicle has its own FBP smart contract with a trained prediction ML model to predict its forwarding unavailability during the upcoming trip. The on-chain prediction model receives the contextual and trip-related data as input and records on-chain the forwarding unavailability probability of the vehicle during its trip. The generated forwarding unavailability probability reflects the expected availability and willingness of the vehicle to participate in data packet forwarding in the upcoming trip.
- **Prediction Registry Smart Contract:** is responsible for maintaining the latest forwarding unavailability predictions by the vehicle nodes. Also, it keeps the FBP smart contract addresses of each participating vehicle in addition to their data normalization parameters. It acts as the pre-routing management component of the MeshChain framework [3], which is a comprehensive blockchain-based framework for managing routing requests, routing validation, consensus, and incentive-based payments in commercialized mesh networks.

For a vehicle to join the IoV network, it should be registered in the Meshchain and linked to a trained ML model embedded on-chain in the Forwarding Behavior Prediction (FBP) smart contract. The vehicles create their own FBP smart contracts and register the address in the Prediction Registry smart contract. Prior to the trip start, the vehicle invokes its own FBP smart contract by passing the temporal, contextual, and spatial input features. The FBP smart contract executes the on-chain trained ML model, generates the forwarding unavailability probability during the upcoming trip, and records it in the Prediction Registry. Other vehicles in the network retrieve the predictions from the Prediction Registry to use them as part of the routing protocol forwarding decisions to build the routing table and select the active relays. To support transparent validation of the on-chain recorded forwarding unavailability probability, the vehicles share their trip spatial information: the trip's start and end locations, while the temporal and contextual features are publicly available. Hence, any vehicle can validate the announced prediction probability by retrieving the FBP smart contract address of the triggered vehicle from the Prediction Registry, executing the on-chain prediction function, and comparing the outcome with the recorded probability.

The predicted forwarding unavailability probability of each vehicle is integrated into the QoS-OLSR routing protocol to prefer more stable relays and forwarders that are available and willing to forward data packets, hence improving the service continuity and avoiding penalizing unavailable vehicles due to contextual reasons. Once vehicles retrieve their neighbors' forwarding unavailability probabilities, the availability decision is made based on a threshold that defines the maximum acceptable probability of a vehicle becoming unavailable for routing participation. The availability decision becomes part of the relay selection and route establishment. The routing protocol drops the Quality-of-Service score to avoid being used as a relay and increases the effective link costs connecting to the predictably unavailable vehicle.

4. Forwarding unavailability prediction model

4.1. Features description

The weather contextual and trip spatial features are important factors that affect the decision of vehicles to provide service [5]. However, publicly available IoV datasets that jointly contain trip-level context, weather conditions, and explicit forwarding availability labels are limited. Therefore, this work uses the Uber Rides Peru dataset [40] as a proxy behavioral dataset to evaluate the feasibility of context-aware forwarding availability prediction. The dataset includes temporal, spatial, and contextual features for 10,551 trips performed by 18 different drivers, where each trip is labeled as completed or cancelled.

It is important to clarify that the dataset does not directly record packet-forwarding decisions. Therefore, the learning target in this work is not packet dropping behavior itself. Instead, the model predicts the probability that a rational vehicle becomes unavailable to provide a crowdsourced service during its trip. Under this abstraction, ride cancellation is not assumed to be identical to packet-forwarding refusal. Rather, both are modeled as forms of context-dependent withdrawal

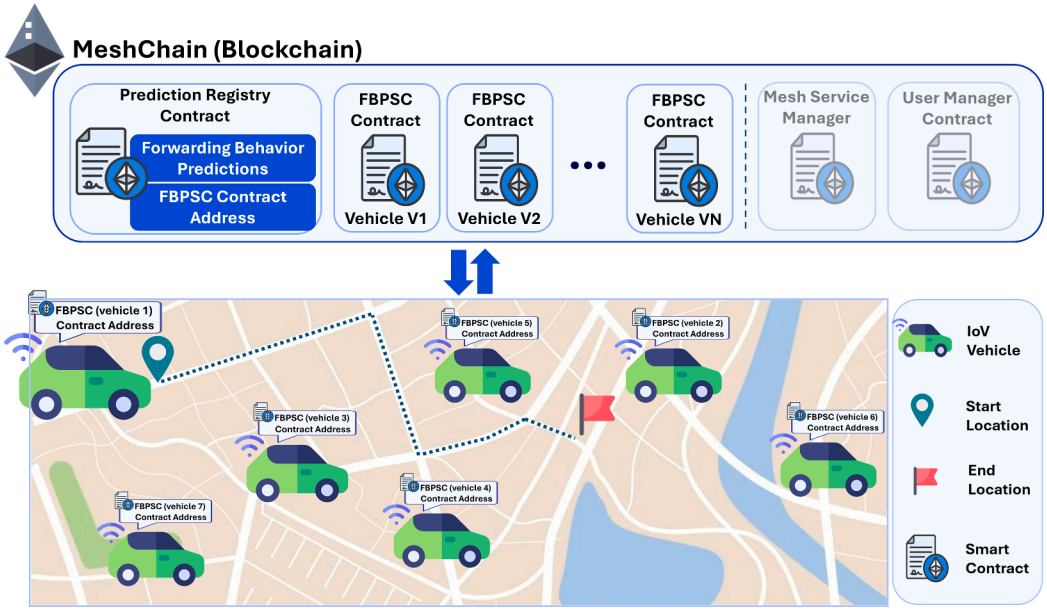


Fig. 1. System Model Overview.

from a committed service by rational mobile participants, even though the operational services differ. Accordingly, completed trips are mapped to the available class, while cancelled trips are mapped to the unavailable class. The drivers in the dataset are therefore mapped to rational vehicles participating in the crowdsourced IoV network.

The target and features used in this work are presented in Table 2. In the considered crowdsourced IoV environment, participating vehicles are rational and independently owned nodes that voluntarily contribute their devices and communication resources to support packet forwarding. Therefore, their forwarding availability is not determined only by network-level conditions, but may also depend on their trip characteristics, individual preferences, and surrounding context. Temporal features include the hour of the day, day of the week, and whether the trip occurs during a weekend. These features capture daily and weekly variations in user behavior and service willingness. For example, some users may be more willing to provide service during weekdays than weekends, leading to a higher probability of unavailability during weekends. Spatial features include the start and end locations of the vehicle's trip, which capture the influence of trip region and route context. Contextual features represent the weather conditions during the trip, since environmental conditions may affect the user's willingness or ability to keep the device available for forwarding. Some weather features are binary, such as whether the weather is rainy, clear sky, light clouds, or heavy clouds, while other contextual features are continuous, including temperature, wind speed, wind direction, and cloud percentage.

4.2. Data Pre-Processing

The pre-processing consists of two main stages, namely, class balancing and feature normalization. The balancing stage is applied to address the class imbalance between the two forwarding availability classes, namely, available and not available, while the normalization stage is applied to transform the training features into comparable numerical values before model training. The pre-processing is implemented on a vehicle-wise basis, where each vehicle file is processed independently to preserve the local trip behavior of the vehicle.

4.2.1. Class balancing using SMOTE

The original vehicle-level trip datasets exhibit class imbalance between vehicles that are available and not available for forwarding during the trip. In particular, the Uber Rides Peru dataset is highly unbalanced, where the ratio of positive to negative samples is 196 : 10355, such that the positive samples represent only 1.89% of the dataset. In this work,

Algorithm 1 Data Balancing using SMOTE Algorithm.

Input: Vehicle's i trip dataset (D_i), features set (F)
Output: Balanced vehicle trip dataset ($\hat{D}_i$)
Initialize:
$X \leftarrow \emptyset$, $y \leftarrow \emptyset$, $D_{valid} \leftarrow \emptyset$, $D_{syn} \leftarrow \emptyset$

for each trip $t \in D_i$ **do**
  if t.cancel_flag $= 0$ **then**
    $unavaiability_label \leftarrow 0$
  else
    $unavaiability_label \leftarrow 1$
  if $t[F]$ has no missing values **then**
    $X \leftarrow X \cup t[F]$
    $y \leftarrow y \cup label$
    $D_{valid} \leftarrow D_{valid} \cup t$
$n_{\min} \leftarrow |\{j \mid y_j = 1\}|$
if $n_{\min} < 2$ **then**
  for each trip $t \in D_i$ **do**
    t.real_data $\leftarrow 1$
  $\hat{D}_i \leftarrow D_i$
else
  $k \leftarrow \min(5, n_{\min} - 1)$
  $(X_{res}, y_{res}) \leftarrow \mathrm{SMOTE}(X, y, k)$
  $n_{orig} \leftarrow |X|$
  for $j \leftarrow n_{orig}$ to $|X_{res}| - 1$ **do**
    Create synthetic trip t_s
    $t_s[F] \leftarrow X_{res}[j]$
    t_s.unavaiability_label $\leftarrow 1$
    t_s.node_id $\leftarrow D_i[0]$.node_id
    t_s.real_data $\leftarrow 0$
    $D_{syn} \leftarrow D_{syn} \cup t_s$
  for each trip $t \in D_i$ **do**
    t.real_data $\leftarrow 1$
  $\hat{D}_i \leftarrow D_i \cup D_{syn}$
Re-index $\hat{D}_i$.trip_id from 1 to $|\hat{D}_i|$

Table 2
Summary of features and data types.

Feature	Type	Description
Temporal Features		
Hour	Integer [0–24]	Hour of the day
Week day	Integer [0–6]	Day of the week encoded into integer value
Weekend	Binary	Whether the day is a weekend
Spatial Features		
Trip start X-location	Integer [0–100]	X coordinates of the start location of the trip
Trip start Y-location	Integer [0–100]	Y coordinates of the start location of the trip
Trip end X-location	Integer [0–100]	X coordinates of the end location of the trip
Trip end Y-location	Integer [0–100]	Y coordinates of the end location of the trip
Contextual Features		
Rainy	Binary	Whether it is a rainy day
Clear clouds	Binary	Whether the sky is clear during the trip
Light clouds	Binary	Whether the sky was lightly cloudy during the trip
Heavy clouds	Binary	Whether the sky was heavily cloudy during the trip
Temperature	Float	Temperature value during the trip (K)
Feels like	Float	Feels-like temperature during the trip (K)
Pressure	Float	Air pressure during the trip (hPa)
Humidity	Float	Level of humidity during the trip (%)
Dewpoint	Float	Dew point temperature on that day (K)
Wind speed	Float	Speed of the wind during the trip (m/s)
Wind direction	Float	Direction of the wind during the trip (deg.)
Cloud percentage	Float	Percentage of clouds during the trip (%)
Target		
Forwarding unavailability	Float	Probability of a vehicle being not available to forward data packets during the trip

the target label represents the forwarding unavailability of the vehicle during the trip. The label is derived from the trip status in the Uber Rides Peru dataset, where completed trips are mapped to the available class and canceled trips are mapped to the not available class. Accordingly, trip cancellation is interpreted as forwarding unavailability in the IoV environment, where vehicles are rational nodes that can decide to stop providing forwarding service during the trip. To address this issue, the Synthetic Minority Over-sampling Technique (SMOTE) is applied independently to each vehicle's data as presented in Algorithm 1.

In the implementation, the source label is obtained from the cancel flag column. A trip with $cancel\ flag = 0$ is mapped to the available class, while a trip with $cancel\ flag = 1$ is mapped to the not available class. The features used for SMOTE are the trip start and end coordinates, temporal context, and weather-related attributes that are presented in Table 2. Rows containing missing values in any of these features are excluded from the SMOTE stage to ensure valid interpolation in the feature space.

SMOTE is applied only when the minority "not available" class contains at least two valid samples. For each vehicle, the number of nearest neighbors k is selected adaptively as presented in Eq. (1).

$$k = \min(5, n_{\min} - 1) \tag{1}$$

where $n_{\min}$ is the number of minority class samples in the vehicle data.

After oversampling, the synthetic minority samples generated by SMOTE are reconstructed as synthetic "not available" trips. The synthetic rows are then assigned the interpolated spatial, temporal, and weather attributes, and are explicitly mapped to the not available class. Also, the vehicle identifier is preserved from the original vehicle data.

4.2.2. Standard normalization

In the second stage of pre-processing, after class balancing, the vehicles' trip data are normalized using standard normalization, since several input features are continuous numerical values with different units, scales, and ranges. Standard normalization is adopted because it centers each feature around zero and scales it by its standard deviation, making the continuous features comparable without depending on fixed minimum and maximum values. In the implementation, the generated binary label `forwarding_unavailability` is used as the source label. The normalized feature set consists of `start_x`, `start_y`, `end_x`, `end_y`, `hour`, `week_day`, `weekend`, `rain`, `clearsky`, `lightclouds`, `heavyclouds`, `temperature`, `feelslike`, `pressure`, `humidity`, `dewpoint`, `windspeed`, `winddir`, and `cloudperc`.

The binary features `weekend`, `rain`, `clearsky`, `lightclouds`, and `heavyclouds` are kept unchanged and are not standardized. However, the remaining non-binary features are normalized using `StandardScaler` before being used for ML model training. For each feature x, the normalized value x' is computed according to Eq. (2).

$$x' = \frac{x - \mu_x}{\sigma_x} \tag{2}$$

where μ_x and σ_x denote the mean and the standard deviation of feature x, respectively, computed from the corresponding vehicle training file. This transforms the features into comparable numerical values and prevents features with larger ranges from dominating the model training process. The mean and scale values of the standardized features are saved for each vehicle. This allows the same normalization parameters to be recorded on-chain and reused during evaluation and deployment.

4.3. Lightweight XGBoost for forwarding unavailability prediction

In this section, the tree-based forwarding unavailability prediction models are discussed. Each vehicle trains its own model off-chain, then deploys it and predicts the forwarding unavailability on-chain. The adopted model in this work is a lightweight XGBoost model, where each vehicle's separate model is trained on the pre-processed normalized vehicle data. Although a larger number of decision trees generally improves the prediction performance by increasing the modeling capacity, the larger XGBoost model, with 300 trees and maximum depth equal to 5, is used as a reference model; however, the lightweight XGBoost model is adopted as it is suitable for on-chain implementation.

The training and deployment of the lightweight XGBoost model happen on 2 stages: 1) feature Pruning, 2) model training per vehicle, and 3) converting the trained model to an on-chain deployable smart con-

tract. Algorithm 2 presents the steps to generate the lightweight XGBoost model per vehicle off-chain.

The first step, **feature pruning** of each vehicle model, aims to preserve the most relevant features for its own prediction model to minimize the number of passed features to the on-chain deployed model and hence reduce the gas cost. Therefore, to determine the final input features of each vehicle predictor, a feature pruning stage is applied to each vehicle's data before training the final adopted model. In this stage, a 5-fold GroupKFold is used as the outer evaluation setting, while a GroupShuffleSplit with validation size equal to 0.2 is used inside each training fold to construct the validation set. For each fold, a lightweight XGBoost model with 10 trees and depth 3 is first trained on the full feature set. Then, the permutation importance of each feature I_{f_j} is computed on the validation set using average precision as the scoring metric and 5 repetitions. For each vehicle, the most frequent subset type is first identified. Then, the features that appear at least three times across the selected fold-wise subsets are retained as the recommended features. If no feature satisfies this condition, the most frequent features are selected according to the median subset size of that vehicle. The permutation importance of each feature I_{f_j} is computed as the difference between the validation average precision after randomly shuffling the values of f_j across the validation samples, while keeping the remaining features unchanged, as presented in Eq. (3). This breaks the relation between that feature and the target label, and the higher the decrease in the validation performance, the higher the importance of the feature to the model.

$$I(f_j) = \frac{1}{R}\sum_{r=1}^{R}\Big(Precision(D_i^{val}) - Precision(D_{i,j,r}^{perm})\Big), \tag{3}$$

where $Precision(D_i^{val})$ is the average precision on the original validation set, $Precision(D_{i,j,r}^{perm})$ is the average precision after permuting feature f_j in repetition r, and $R = 5$ is the number of repetitions. Also, $D_i^v al$ stands for the validation set of vehicle i, while $D_{i,j,r}^{perm}$ is the validation set after permuting feature f_j in repetition r.

After $I(f_j)$ computation for all the features, they are then ranked in descending order to obtain $Rank_i^f$. Then, multiple candidate feature subsets are evaluated with the number of features $N_{features} \in \{4, 6, 8, 10, 19\}$, where $N_{features} = 19$ presents the full feature set. The subset with the highest validation F1-score is selected, while smaller subsets are preferred in case of ties. The F1-score is used as the selection metric since the forwarding unavailability prediction problem is imbalanced, and the objective is to balance the precision and recall of detecting not-available vehicles rather than being biased towards the majority class. The final recommended feature set F_i^{sel} of each vehicle is determined from the selected subsets across the folds.

The second stage is **training the lightweight XGBoost model** using the vehicle's recommended pruned feature set. The lightweight XGBoost model is configured with 10 trees, maximum depth equal to 3, learning rate equal to 0.3, full row and feature sampling, logistic binary objective, and histogram-based tree construction. The number of trees and depth are chosen to reduce complexity and execution cost on-chain. Although the class imbalance is addressed in the data pre-processing stage using SMOTE, the lightweight XGBoost model further supports the handling of the remaining imbalance during training using a $scalePosWeight$ value computed as the ratio between the negative and positive samples of the corresponding vehicle. The output of the trained model of vehicle i is the probability that the vehicle is not available for forwarding during the trip.

The lightweight XGBoost model outputs a probability for the forwarding unavailability of the vehicle during the trip. Since the dataset label used in training and evaluation is binary, a prediction threshold θ_i is required to convert the probability output into a binary decision. For evaluation, the prediction threshold is not fixed for all vehicle models. Instead, the validation probabilities are used to tune the decision threshold θ_i over the range from 0.05 to 0.95 with a step of 0.01. The threshold that maximizes the validation F1-score is selected. The selected threshold θ_i is used first to compare the candidate feature subsets during the pruning stage, and then it is applied to the predicted probabilities of the test fold to evaluate the selected subset.

Finally, the trained lightweight XGBoost model of each vehicle is exported for deployment and **converted into an executable smart contract** for on-chain forwarding unavailability prediction. The model metadata, including the indices of the selected pruned feature subset F_i^{sel} and the scaling factor α_i, are also recorded on-chain for each vehicle i. This allows any IoV network node to construct the required input vector and call the prediction functions of the smart contract of the corresponding vehicle. In addition, the continuous input features and the output prediction probability are scaled using α_i to integer values, as the floating-point processing incurs high gas cost and the integer computations are more efficient for on-chain execution.

Algorithm 2 Vehicle Lightweight XGBoost Training and Export Algorithm.

Input: Normalized vehicle dataset (D_i), full feature set (F)
Output: Final model (M_i), selected feature subset (F_i^{sel}), scaling factor (α_i)
Initialize: $S_i \leftarrow \emptyset$, $C \leftarrow \{4, 6, 8, 10, 19\}$
for each fold $fd \in \{1, \ldots, 5\}$ **do**
    Split D_i into $D_i^{trainval}$ and D_i^{test} using `GroupKFold`
    Split $D_i^{trainval}$ into D_i^{train} and D_i^{val} using `GroupShuffleSplit`
    Train lightweight XGBoost model M_i^{fd} on D_i^{train} using F
    Compute features importance I_{f_j} using Eq. (3), for each $f_j \in F$,.
    Rank the features in descending order of $I_{i,j}^{fd}$ to obtain $Rank_i^{fd}$
    for each $n \in C$ **do**
        $F_{i,n}^{fd} \leftarrow$ top-n ranked features from R_i^f
        Train model $M_{i,n}^{fd}$ on D_i^{train} using $F_{i,n}^{fd}$
        Predict validation probabilities $P_{i,n}^{val}$
        Tune threshold $\theta_{i,n}^{fd}$ over $[0.05, 0.95]$ with step 0.01
        Compute validation F1-score $F1_{i,n}^{fd}$ using $\theta_{i,n}^{fd}$
    Select $F_i^{f,*}$ with maximum $F1_{i,n}^{fd}$
    if multiple subsets have the same maximum F1-score **then**
        Select the subset with smaller size
    $S_i \leftarrow S_i \cup F_i^{fd,*}$
Determine the most frequent subset type in S_i
Construct F_i^{sel} using features repeated at least 3 times across folds
if $F_i^{sel} = \emptyset$ **then**
    Select the most frequent features according to the median subset size
Compute $scalePosWeight_i \leftarrow \dfrac{N_i^-}{N_i^+}$
Train final lightweight XGBoost model M_i on D_i using F_i^{sel}
Set scaling factor α_i
Scale continuous input features and output probability using α_i
Export M_i and convert it into executable smart contract logic
Record on-chain the indices of F_i^{sel} and the scaling factor α_i

The deployed lightweight XGBoost model of each vehicle is executed on-chain through the Forwarding Behavior Predictor smart contract, and the implementation is presented in Algorithm 3. The contract receives the scaled selected input features of the vehicle, where the order of the features is determined by the selected feature subset $\mathcal{F}_i^{sel}$. Then, the contract computes the raw prediction score by aggregating the outputs of the K generated tree functions of the lightweight XGBoost model. Afterward, the raw score is transformed into a forwarding unavailability probability using the implemented piecewise linear sigmoid approximation. This approximation is fixed and shared by all generated predictor contracts and is used to avoid the expensive exponential and floating-point computations in smart contracts. The exact forwarding unavailability probability p_i computed using the sigmoid function is pre-

sented in Eq. (4).

$$p_i = \frac{1}{1 + e^{-s_i}}. \tag{4}$$

where s_i denote the raw prediction score of vehicle i.

However, the sigmoid function implemented on-chain is approximated using the piecewise linear function to compute the approximated forwarding unavailability probability $\hat{p}_i(s_i)$ with a scaling factor S as presented in Eq. (5). The scaling factor is used to avoid floating-point computation in smart contracts. -

$$\hat{p}_i(s_i) = \begin{cases} 0, & s_i \le -6S, \\ y_1 + \dfrac{(s_i + 6S)(y_2 - y_1)}{3S}, & -6S < s_i < -3S, \\ y_2 + \dfrac{(s_i + 3S)(y_3 - y_2)}{2S}, & -3S \le s_i < -S, \\ y_3 + \dfrac{(s_i + S)(y_4 - y_3)}{2S}, & -S \le s_i < S, \\ y_4 + \dfrac{(s_i - S)(y_5 - y_4)}{2S}, & S \le s_i < 3S, \\ y_5 + \dfrac{(s_i - 3S)(y_6 - y_5)}{3S}, & 3S \le s_i < 6S, \\ 1, & s_i \ge 6S, \end{cases} \tag{5}$$

where y_i , $i \in \{1, ..., 6\}$ are defined as follows,

$y_1 = 0.0025$, $y_2 = 0.0474$, $y_3 = 0.2689$, $y_4 = 0.7311$, $y_5 = 0.9526$, $y_6 = 0.9975$.

These values correspond to the sigmoid at the fixed operating points −6, −3, −1, 1, 3, and 6, respectively. The resulting probability is then scaled using the scaling factor α_i as presented in Eq. (6).

$$FW\,Prob_i^{scaled} = \lfloor \alpha_i \hat{p}_i(s_i) \rceil \tag{6}$$

The scaled forwarding unavailability probability $prob_i^{scaled}$ is then uploaded to the Prediction Registry, where it becomes publicly available to the IoV network.

Algorithm 3 On-Chain Forwarding Unavailability Prediction Algorithm.

Function *predictAndUpload()* external:
Input: $user$, $features$
Require $user \neq 0$
Require $size(features) = size(\mathcal{F}_i^{sel})$
Require Prediction Registry is set
Initialize: $score_i \leftarrow 0$
for $k = 0$ to $K - 1$ **do**
    $score_i \leftarrow score_i + _tree_k(features)$
$prob_i^{scaled} \leftarrow _sigmoidApprox(score_i)$
Call *addPredictionFromContract()* in Prediction Registry and pass $user$, $prob_i^{scaled}$
Emit *PredictionUploaded*($user$, $prob_i^{scaled}$)

Function *_tree$_k$()* internal:
Input: $features$
Output: $treeScore_k$
Evaluate the decision conditions of tree k using the selected feature values in $features$ in nested if, else structure
Return the leaf score of the reached decision path

Function *_sigmoidApprox()* internal:
Input: s_i
Output: $prob_i^{scaled}$
Compute $\hat{p}_i(s_i)$ according to Eq. (5)
$prob_i^{scaled} \leftarrow \lfloor \alpha_i \hat{p}_i(s_i) \rceil$

5. Forwarding Behavior-Aware QoS-OLSR routing protocol

The forwarding unavailability prediction is integrated into the QoS-OLSR protocol [41] to transform the protocol from reacting only to disconnecting and dropping vehicles into proactively avoiding vehicles that are predicted to become unavailable during their trip. In the implemented routing protocol, each vehicle retrieves the forwarding unavailability predictions of its neighboring vehicles and uses them as part of the routing decision process. The integration is applied using the predicted forwarding unavailability probability $FW\,Prob_j$ of a neighbor vehicle j and a prediction threshold $\theta_{unavailable}$, where a neighbor vehicle j is predicted unavailable when $FW\,Prob_j > \theta_{unavailable}$. To avoid unavailable nodes, the protocol 1) updates the vehicle's willingness and QoS, and 2) modifies the routing table computation by updating the link cost of predicted unavailable vehicles. The willingness and QoS of the predicted unavailable vehicle are updated, which reduces its role in the selection of cluster heads and MPRs. Also, the cost of all the links connecting to the predicted unavailable vehicle is updated, which affects the routing table computation and prefers routing paths away from unavailable neighbors. Accordingly, unavailable vehicles are proactively avoided in the dissemination and forwarding of both data and control packets, which improves the performance of QoS-OLSR under the dynamic conditions of the IoV environment.

First, the prediction is integrated into the `HELLO` message and QoS computation. In the `HELLO` message, if the vehicle i itself is predicted unavailable, it sets its willingness to 0, i.e. not willing to route, and the QoS value is set to 0, then it is appended to the `HELLO` message to broadcast it to the network. The QoS calculation, presented in Eq. (7), affects the MPR selection, which are the core network nodes in the QoS-OLSR protocol responsible for control and data packets dissemination.

$$QoS_i = \begin{cases} 0, & FW\,Prob_i > \theta_{unavailable}, \\ 100 \times \left(2 - atan\left(Avg(LinksCost_i) - 4\right)\right) & \text{otherwise}, \end{cases} \tag{7}$$

where the QoS value for available nodes is calculated based on the QoS-OLSR protocol proposed in [41]. It is computed as a function of the average of all the link costs of vehicle i. Therefore, a vehicle predicted to become unavailable announces itself as unwilling to carry and forward data traffic and advertises no routing quality to its neighbors. Consequently, such a vehicle is avoided during the cluster heads and MPR selection procedures of QoS-OLSR. In addition to self-advertisement of unavailability, each vehicle retrieves the $FW\,prob_j$ from the Prediction Registry smart contract for each neighbor $j \in NB_i$ and compares it against the announced unavailability.

Second, the prediction $FW\,prob_j$ is integrated into routing table computation, such that the cost of the link connecting to the predicted unavailable neighbor j $linkCost_{ij}$ is set to a very high cost and considered broken. Hence, the link between vehicle i and unavailable neighbor j is avoided in the routing table computation. This is achieved by setting the link quality to 0 since it is a core element in the link cost $linkCost_{ij}$ computation as presented in Eq. (8).

$$linkCost_{ij} = \begin{cases} M, & FW\,Prob_j > \theta_{unavailable}, \\ \{LQ_{ij} \times LQ_{ji}\}^{-1} \times sigPenalty & \text{otherwise}, \end{cases} \tag{8}$$

where M is a very large number that indicates a disconnected link and $sigPenalty$ is a penalty value between 0 and 1 based on the sensed signal strength obtained from the WiFi device. LQ_{ij} stands for the link quality between vehicles i and j. It is computed as a ratio of received to expected to be received control packets within a window of time.

Table 3
Forwarding Behavior Predictor smart contract of a vehicle.

Variables			
Prediction Registry Interface (interface)			
Function	**Type**	**Parameter**	**Return**
expectedFeatureCount()	External	–	Selected Features Count
getSelectedFeatures()	External	–	Selected Feature List [ ]
setPredictionRegistry()	External	Prediction Registry Address	–
predictRawScore()	Public	Features [ ]	Raw Prediction Score
predictAndUpload()	External	User, Features [ ]	Scaled Prediction Probability
_sigmoidApprox()	Internal	Raw Prediction Score	Scaled Prediction Probability
$_tree_k()$, $\forall k \in \{0, ..., K-1\}$	Internal	Features [ ]	Tree Contribution Score
Events	**Parameters**		
PredictionUploaded	User, Scaled Prediction Probability		

6. Smart contracts implementation

The smart contracts, Forwarding Behavior Predictor and the Prediction Registry, are responsible for performing the prediction in a transparent and immutable manner, in addition to providing the network with a mechanism to disseminate forwarding unavailability predictions and the parameters necessary for predictions' validations.

The **Forwarding Behavior Predictor (FBP_i)** smart contract is responsible for executing the forwarding unavailability prediction model of vehicle i and uploading the generated prediction to the Prediction Registry for access by all IoV network nodes. Each vehicle in the network has its own FBP. The contract's variables, functions, and events are presented in Table 3. The *Prediction Registry Interface* variable is the interface used to communicate with the Prediction Registry contract and upload the generated predictions for vehicle i. The interface calls *addPredictionFromContract()* defined in the Prediction Registry contract. The function *expectedFeatureCount()* returns the number of input features required by the predictor contract, while *getSelectedFeatures()* returns the ordered list of the selected features used by the prediction model. The address of the Prediction Registry is assigned using the *setPredictionRegistry()* function, which enables the predictor contract to upload the prediction output on-chain. The function *predictRawScore()* receives the input feature vector and computes the raw prediction score of the lightweight XGBoost model. Internally, the contract includes the generated tree functions $_tree_k()$, where $k \in \{0, ..., K-1\}$ and K is the total number of trees. Each tree function represents one decision tree of the model and returns its contribution to the final raw score. Also, the function *_sigmoidApprox()* converts the aggregated raw score into a scaled prediction probability. The function *predictAndUpload()* receives the user address and the input feature vector, computes the scaled forwarding unavailability probability, and uploads it to the Prediction Registry. After the prediction is uploaded, the *PredictionUploaded* event is emitted.

The **Prediction Registry** contract, presented in Table 4, is responsible for storing the forwarding unavailability predictions and the parameters required for their interpretation and validation. The contract comprises three main data structures, namely, *UserConfig*, *PredictionPoint*, and *PredictionBuffer*. The *UserConfig* data structure contains the configuration of a vehicle predictor, including the *Node ID*, the *Scaling Factor*, the *Selected Feature Indices*, the *Normalization Means*, the *Normalization Scales*, the *Updated Time*, and the *Exists* flag. The *Node ID* identifies the corresponding vehicle, while the *Scaling Factor* is used to represent the continuous features and the prediction probability as integer values for efficient on-chain computation. The *Selected Feature Indices* determine the order of the pruned input feature subset required by the predictor contract. Also, the *Normalization Means* and *Normalization Scales* contain the parameters used to normalize the input features before prediction. The *PredictionPoint* data structure stores one prediction value as the *Scaled Prediction Probability* and its associated *Timestamp*. The *PredictionBuffer* data structure maintains the prediction history of a user using a fixed-size buffer, in addition to the current *Count* of stored predictions and the *Next Index* to write.

The variables of the Prediction Registry include mappings,*userConfigs*, *predictionHistory*, and *nodePredictorByNodeId*. The *userConfigs* variable is a mapping of the user address to its *UserConfig* structure. The *predictionHistory* variable is a mapping of the user address to its *PredictionBuffer* structure, where the predictions of the user are recorded. Also, the *nodePredictorByNodeId* variable is a mapping of the node ID to the address of its corresponding Forwarding Behavior Predictor contract.

The functions of the Prediction Registry are responsible for registering the predictor contracts, updating the users' prediction configuration, storing the uploaded predictions, and retrieving the recorded predictions and configurations. The function *setNodePredictor()* assigns the predictor contract address to a specific node ID. The function *upsertMyConfig()* is used by the user to insert or update its prediction configuration, including the scaling factor, selected feature indices, and normalization parameters. The function *addPredictionFromContract()* is called by the authorized predictor contract to add a new prediction of a user to the registry. The configuration of the caller can be retrieved using *getMyConfig()*, while the configuration of any user can be retrieved using *getUserConfig()* by passing the user address. Also, the latest prediction of a user can be retrieved using *getLatestPrediction()*, while the total number of stored predictions is returned by *getPredictionCount()*. In addition, *getPredictionHistory()* returns the full stored prediction history of the user, and *getLatestPredictions()* returns the latest recorded predictions up to a requested limit.

The Prediction Registry also emits a set of events to announce the updates performed on-chain. The *NodePredictorSet* event is emitted when a predictor contract is assigned to a node ID. The *UserConfigUpdated* event is emitted when the configuration of a user is inserted or updated. Also, the *PredictionAdded* event is emitted whenever a new forwarding unavailability prediction is added to the registry. Therefore, the Prediction Registry contract provides the IoV network with a transparent and immutable mechanism to disseminate the forwarding unavailability predictions and the parameters necessary for prediction validation.

Fig. 2 presents the smart-contract interaction workflow used to generate and validate the forwarding unavailability prediction. In the first phase, the vehicle deploys its vehicle-specific Forwarding Behavior Predictor contract, links it to the Prediction Registry, and stores the required configuration parameters, including the selected feature indices, scaling factor, and normalization values. In the second phase, the vehicle invokes the predictor before the trip by submitting the scaled input features, after which the contract evaluates the lightweight XGBoost model, computes the scaled forwarding unavailability probability, and uploads the result to the Prediction Registry. In the third phase, a neighboring vehicle retrieves the stored prediction and the associated configuration parameters from the registry, re-executes the predictor contract using the reconstructed input features, and compares the recomputed output

Table 4
Prediction Registry smart contract.

Data Structures		
UserConfig		
Node ID (uint256)	Scaling Factor (uint256)	Selected Features (uint256[])
Normalization Means (int256[])	Normalization Scales (uint256[])	Updated Time (uint256)
Exists (bool)		
PredictionPoint		
Timestamp (uint256)	Prediction Probability (uint256)	
PredictionBuffer		
Count (uint256)	Prediction History (PredictionPoint[])	Next Index (uint256)
Variables		
userConfigs (address ⇒ UserConfig)		
predictionHistory (address ⇒ PredictionBuffer)		
nodePredictorByNodeId (uint256 ⇒ address)		
Function	**Parameter**	**Return**
setNodePredictor()	Node ID, Predictor Address	–
upsertMyConfig()	UserConfig	–
addPredictionFromContract()	User, Scaled Prediction Probability	–
getMyConfig()	-	UserConfig
getUserConfig()	User	UserConfig
getLatestPrediction()	User	Predictions, Timestamp
getPredictionCount()	User	Prediction Count
getPredictionHistory()	User	Predictions [], Timestamps []
getLatestPredictions()	User, Limit	Predictions [], Timestamps []
Events	**Parameters**	
NodePredictorSet	Node ID, Predictor Address	
UserConfigUpdated	User, Node ID, Scaling Factor, Updated Time	
PredictionAdded	User, Prediction, Timestamp	

with the recorded value to validate the prediction before using it in routing decisions.

7. Implementation results

In this section, the proposed work evaluation is discussed. The XGBoost model performance is compared to the baselines and discussed in Section 7.1. The integration of unavailability predictions into the network is evaluated in Section 7.2. While, the gas cost of the smart contracts implementation of the Prediction Registry and the FBP is presented in Section 7.3.

7.1. ML model performance

In this section, the performance of the forwarding unavailability prediction models is discussed. The proposed lightweight XGBoost is compared to the baselines, XGBoost with 300 trees and depth 3, in addition to the logistic regression. The baselines are discussed as follows,

- **Reference XGBoost model** is used as the reference tree-based model for performance comparison. It is implemented with 300 trees, a maximum depth equal to 3, a learning rate equal to 0.3, full row sampling, full feature sampling, and a logistic binary objective. Also, the imbalance between the classes is handled using the *scalePosWeight* parameter computed from the training data of each vehicle.
- **Logistic Regression** model is used as a linear baseline for forwarding unavailability prediction. It is implemented using L1 regularization with $C = 0.2$, the *liblinear* solver, balanced class weighting, and a maximum of 2000 iterations.
- **Random Forest model** is used as a bagging-based tree ensemble baseline. It is implemented with 100 decision trees, maximum depth equal to 8, balanced class weighting, and square-root feature sampling. This model evaluates whether averaging multiple independent decision trees improves forwarding unavailability prediction.
- **Gradient Boosting model** is used as another boosted tree ensemble baseline. It is implemented with 100 estimators, learning rate equal to 0.05, maximum depth equal to 3, and subsampling ratio equal to 0.8. This model evaluates the performance of a conventional gradient-boosted tree classifier under the same vehicle-wise evaluation setting.
- **AdaBoost Stumps model** is used as a lightweight boosting baseline based on shallow weak learners. It is implemented with 100 depth-one decision stumps and learning rate equal to 0.05. This model evaluates whether a low-depth boosted ensemble can capture the forwarding unavailability behavior.
- **MLP Neural Network model** is used as a lightweight neural-network baseline. It consists of one hidden layer with 16 neurons and is trained using the Adam optimizer with early stopping. This model evaluates the suitability of a compact neural classifier for the considered prediction task.
- **LightGBM model** is used as an efficient gradient-boosted tree baseline. It is implemented with 100 estimators, learning rate equal to 0.05, maximum depth equal to 5, subsampling, column sampling, and class-imbalance weighting. This model is included because LightGBM is widely used for efficient tabular-data classification.

7.1.1. Evaluation metrics

The performance of the proposed lightweight XGBoost model is evaluated using accuracy, recall, F1-score, PR-AUC, and Brier score. Since the dataset is imbalanced, the evaluation does not rely only on accuracy but also includes metrics that better reflect the detection performance of the unavailable vehicles and the quality of the predicted probabilities. In this work, the positive class corresponds to the unavailable vehicles.

The following are the ML models' performance evaluation metrics,

1. **Accuracy** is computed as in Eq. (9).

$$Accuracy = \frac{TP + TN}{Number\ of\ test\ samples} \tag{9}$$

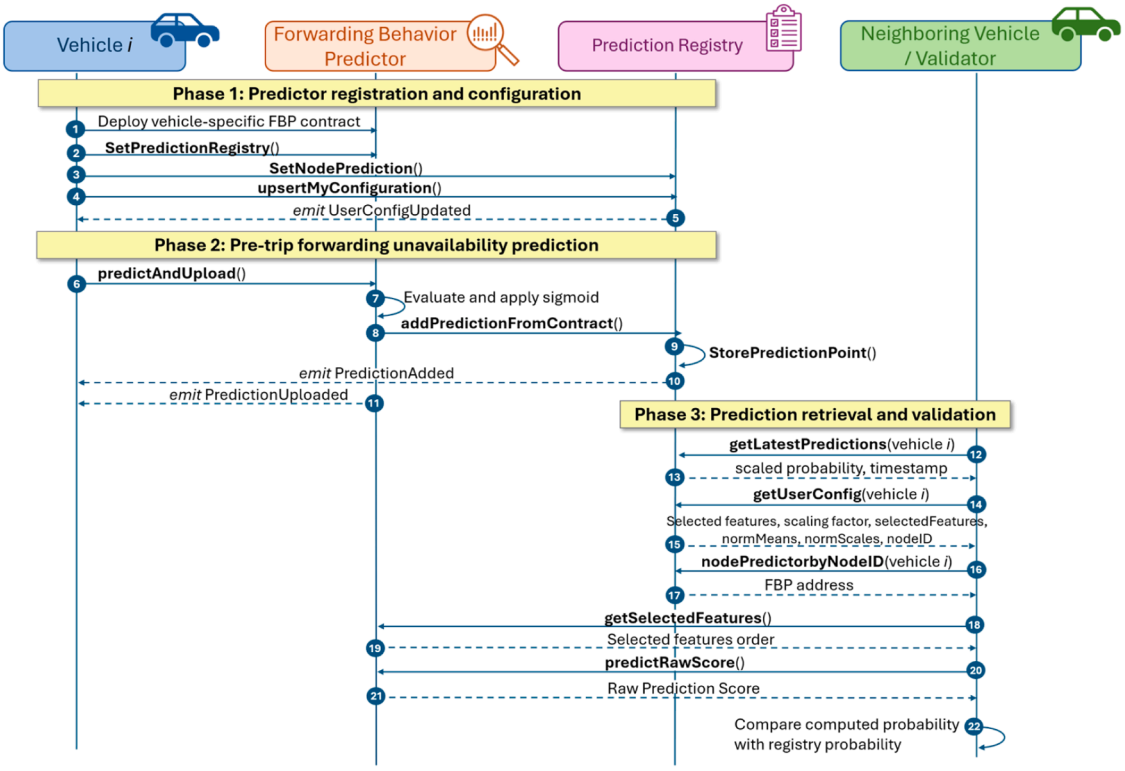


Fig. 2. Sequence diagram of the smart-contract-assisted forwarding unavailability prediction workflow, including predictor registration, pre-trip prediction upload, and prediction retrieval and validation.

Where, TP, TN, FP, and FN are the numbers of true positives, true negatives, false positives, and false negatives samples, respectively.

2. **Recall** is used to evaluate the ability of the model to correctly identify the not available vehicles, and is computed in Eq. (10)

$$Recall = \frac{TP}{TP + FN} \tag{10}$$

3. **F1-score** is used to evaluate the balance between precision and recall in detecting the not available vehicles. The precision and F1-score are computed in Eqs. (11) and (12), respectively.

$$Precision = \frac{TP}{TP + FP} \tag{11}$$

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall} \tag{12}$$

4. **PR-AUC** is used to evaluate the ranking quality of the prediction probabilities over different decision thresholds. It is more suitable than the ROC-AUC for imbalanced prediction problems, since it focuses on the tradeoff between precision and recall of the positive class. The PR-AUC is computed as the area under the precision-recall curve as follows

$$PR\text{-}AUC = \int_0^1 Precision(Recall)\, d(Recall) \tag{13}$$

5. **Brier** score is used to evaluate the quality of the predicted probabilities and presents the mean square error of predicted probabilities. Let N denote the total number of samples, p_i the predicted probability of sample i, and $y_i \in \{0, 1\}$ its true label. Then, the Brier score is computed as

$$Brier = \frac{1}{N} \sum_{i=1}^{N} (p_i - y_i)^2 \tag{14}$$

7.1.2. Performance results and analysis

Table 5 presents the comparison between the proposed lightweight XGBoost model and the additional machine learning baselines. The results show that the proposed lightweight XGBoost model achieves an accuracy of 0.9399, recall of 0.9700, F1-score of 0.9424, PR-AUC of 0.9806, and Brier score of 0.0584. These results confirm that the proposed model substantially outperforms the logistic regression, AdaBoost stumps, and MLP neural-network baselines. Compared with logistic regression, the lightweight XGBoost improves the accuracy from 0.8524 to 0.9399, the F1-score from 0.8643 to 0.9424, and the PR-AUC from 0.8745 to 0.9806. This indicates that forwarding unavailability behavior is better captured by tree-based nonlinear decision rules than by a linear classifier.

The results also show that larger ensemble models, such as Random Forest, Gradient Boosting, LightGBM, and the reference XGBoost model, achieve higher performance than the proposed lightweight XGBoost model. For example, LightGBM achieves an F1-score of 0.9696 and PR-AUC of 0.9937, while the reference XGBoost achieves an F1-score of 0.9710 and PR-AUC of 0.9944. However, these improvements are obtained using larger model structures. To clarify this tradeoff, the structural complexity proxy presented in Table 6 represents the approximate number of model parameters or decision nodes used during inference. For tree-based models, it corresponds to the total number of split nodes and leaf nodes across all trees. For the MLP model, it corresponds to the total number of weights and biases. The structural complexity proxy is reported as the average value over all vehicle-specific models and cross-validation folds. This proxy indicates the relative amount of model logic that must be evaluated and translated into smart contract code. The lightweight XGBoost model has a structural complexity proxy of 112.69 with only 10 trees, while Random Forest, Gradient Boosting, LightGBM, and the reference XGBoost have structural complexity proxies of 4227.38, 1379.18, 2247.27, and 1824.31, respectively. Therefore, although the larger ensemble models provide higher predictive performance, they are less suitable for direct smart-contract deployment because they require more tree evaluations, larger generated contract code, and higher deployment overhead.

Accordingly, the adopted lightweight XGBoost model provides a practical tradeoff between prediction performance and on-chain

Table 5
Summary of forwarding unavailability prediction performance.

Model	Accuracy	Recall	F1-score	PR-AUC	Brier
Lightweight XGBoost (10 × 3)	0.9399	0.9700	0.9424	0.9806	0.0584
Reference XGBoost (300 × 3)	0.9701	0.9852	0.9710	0.9944	0.0199
Logistic Regression	0.8524	0.9342	0.8643	0.8745	0.1142
Random Forest	0.9588	0.9692	0.9593	0.9927	0.0476
Gradient Boosting	0.9642	0.9771	0.9647	0.9937	0.0349
AdaBoost Stumps	0.8388	0.9198	0.8517	0.8525	0.1658
MLP Neural Network	0.8076	0.9218	0.8327	0.8541	0.1434
LightGBM	0.9689	0.9809	0.9696	0.9937	0.0279

Table 6
Model complexity comparison.

Model	Complexity Proxy	Number of Trees	Maximum Depth
Lightweight XGBoost (10 × 3)	112.69	10	3
Reference XGBoost (300 × 3)	1824.31	300	3
Logistic Regression	13.11	0	–
Random Forest	4227.38	100	8
Gradient Boosting	1379.18	100	3
AdaBoost Stumps	300.00	100	1
MLP Neural Network	337.00	0	1
LightGBM	2247.27	100	5

Table 7
Effect of the number of trees on the XGBoost prediction performance.

Model	Accuracy	Recall	F1-score	PR-AUC	Brier
XGBoost (5x3)	0.9126	0.9498	0.9160	0.9483	0.0926
XGBoost (10x3)	0.9399	0.9700	0.9424	0.9806	0.0584
XGBoost (15x3)	0.9531	0.9698	0.9542	0.9880	0.0445
XGBoost (20x3)	0.9599	0.9740	0.9606	0.9911	0.0369
XGBoost (30x3)	0.9654	0.9789	0.9661	0.9934	0.0286
XGBoost (300x3)	0.9701	0.9852	0.9710	0.9944	0.0199

implementation feasibility. The model maintains high recall and F1-score for detecting unavailable vehicles, while preserving a compact structure that can be translated into deterministic smart contract logic. This is important because the on-chain predictor must execute using integer operations, threshold comparisons, additions, and a lightweight sigmoid approximation. Therefore, the model selection is not based only on maximizing prediction accuracy, but on balancing prediction performance, model compactness, and smart-contract execution feasibility.

Table 7 shows the effect of the number of trees on the model performance. The results show that increasing the number of trees improves the performance of the XGBoost model. As presented in Table 7, the 5-tree model achieves an F1-score of 0.9160 and PR-AUC of 0.9483, while the 10-tree model improves the F1-score to 0.9424 and the PR-AUC to 0.9806, which are improved by 2.88% and 3.41%, respectively. The performance gradually increases with the number of trees, and the reference XGBoost model with 300 trees achieves the best performance with an accuracy of 0.9701, recall of 0.9852, F1-score of 0.9710, PR-AUC of 0.9944, and Brier score of 0.0199.

Although the reference XGBoost model provides the highest prediction performance, the lightweight XGBoost model is adopted in this work due to its lower implementation complexity and suitability for on-chain deployment. The 10-tree model achieves strong prediction performance while maintaining a compact structure that can be translated into an executable smart contract. Therefore, it provides a better trade-off between prediction performance and deployment cost, as discussed in Section 7.3, than the larger reference model.

Table 8 presents the pooled confusion matrix of the adopted lightweight XGBoost model using the tuned decision threshold. The model produces 772 false positives and 289 false negatives, corresponding to a false positive rate of 7.45% and a false negative rate of 2.79%. In the routing context, false positives may cause OLSR-ML to avoid usable relays and select longer alternative paths, while false negatives are more harmful because unavailable vehicles may still be selected as relays and drop packets. Therefore, the lower false negative rate supports the reliability-oriented objective of the proposed routing framework.

Table 8
Pooled confusion matrix of the adopted lightweight XGBoost model.

	Predicted Available	Predicted Unavailable
Actual Available	9585	772
Actual Unavailable	289	10,062

Table 9
Comparison of imbalance-handling methods under the standard testing setup.

Method	Accuracy	Recall	F1-score	PR-AUC	Brier
Random oversampling	0.4943	0.0254	0.0479	0.5496	0.4182
SMOTE	0.9339	0.9857	0.9358	0.9831	0.0587
Borderline-SMOTE	0.9661	0.9767	0.9610	0.9864	0.0327
ADASYN	0.9362	0.9822	0.9376	0.9844	0.0580

7.1.3. Imbalance handling analysis

To evaluate the effect of imbalance handling on forwarding unavailability prediction, two analyses are conducted. First, different imbalance-handling methods are compared under the standard testing setup to evaluate their overall prediction performance. Second, a real-test-sample reliability analysis is conducted using only original, non-synthetic test samples to examine whether the trained models can detect real unavailable vehicles rather than only performing well on synthetic or oversampled samples.

Four imbalance-handling methods are compared. Random oversampling balances the training data by duplicating existing minority-class samples without generating new data points. SMOTE generates synthetic minority-class samples by interpolating between neighboring minority samples in the feature space. Borderline-SMOTE is a variant of SMOTE that focuses the synthetic sample generation on minority samples located near the decision boundary, where misclassification is more likely. ADASYN adaptively generates more synthetic samples for minority instances that are harder to learn, giving more attention to difficult regions of the feature space.

Table 9 presents the comparison under the standard testing setup. Random oversampling performs poorly, with recall of 0.0254, F1-score of 0.0479, PR-AUC of 0.5496, and Brier score of 0.4182. This indicates that simply duplicating minority samples is not sufficient to learn the rare forwarding-unavailability class in the considered dataset. The three synthetic oversampling methods provide substantially better results. SMOTE achieves high recall of 0.9857, F1-score of 0.9358, PR-AUC of 0.9831, and Brier score of 0.0587. Borderline-SMOTE achieves the highest accuracy, F1-score, PR-AUC, and the lowest Brier score, indicating strong overall predictive performance. ADASYN also provides competitive results, with recall of 0.9822, F1-score of 0.9376, PR-AUC of 0.9844, and Brier score of 0.0580. However, SMOTE achieves the highest recall among the strong-performing methods, which is important for the proposed routing framework because missed unavailable vehicles may remain selected as relays and later cause packet drops. Therefore, SMOTE is adopted as the imbalance-handling method in the proposed framework because it provides a simple and effective reliability-oriented trade-off between detecting unavailable vehicles and maintaining strong overall prediction performance.

For the real-test-sample reliability analysis, a single 80/20 train-test split is used instead of multi-fold cross-validation. This choice is made to preserve a sufficient number of original real unavailable samples in the test set. Since the original dataset contains very few unavailable trips, using several folds would further reduce the number of real positive samples in each test fold and make the reliability analysis highly unstable. Therefore, the one-fold 80/20 split is used only for the real-sample

Table 10
Imbalance-handling comparison on real test samples.

Method	Accuracy	Balanced Accuracy	Recall	Specificity
Random oversampling	94.64%	50.66%	5.00%	96.33%
SMOTE	88.35%	85.48%	82.50%	88.46%
Borderline-SMOTE	94.73%	66.65%	37.50%	95.81%
ADASYN	89.09%	84.63%	80.00%	89.26%

reliability analysis presented in Table 10, while the main model evaluation remains based on the cross-validation setting described earlier.

Table 10 presents the reliability analysis on original, non-synthetic test samples. This analysis examines whether the trained model can detect real unavailable vehicles rather than synthetic or oversampled instances. Random oversampling achieves the highest specificity of 96.33% and high accuracy of 94.64%, but it achieves only 5.00% recall for the unavailable class. This indicates poor reliability for detecting rare forwarding-unavailability events despite the high overall accuracy. In contrast, SMOTE improves the recall of real unavailable vehicles from 5.00% to 82.50% and achieves a balanced accuracy of 85.48%. ADASYN achieves a similar recall of 80.00% and balanced accuracy of 84.63%, while Borderline-SMOTE maintains high accuracy and specificity but achieves lower recall of 37.50%. These results show that SMOTE provides a strong reliability-oriented trade-off for detecting real forwarding-unavailability events. This is important because, in the proposed routing framework, false negatives are more harmful than false positives. A false negative may cause an unavailable vehicle to remain selected as a relay and later drop packets, while a false positive mainly causes the routing protocol to avoid a usable relay and potentially select a longer alternative route.

However, the improvement in unavailable-vehicle recall comes at the cost of lower specificity. Compared with random oversampling, SMOTE reduces specificity from 96.33% to 88.46%, indicating that more available vehicles may be incorrectly predicted as unavailable. Therefore, the real-sample analysis shows a reliability trade-off: SMOTE improves the detection reliability of unavailable vehicles, while random oversampling better preserves available-vehicle classification. This trade-off is consistent with the objective of the proposed routing framework, which prioritizes avoiding unavailable relays. Future work should further investigate threshold tuning and cost-sensitive learning to better balance false negatives and false positives.

7.2. Simulation results

The simulation experiments are carried out using NS-3, and the simulations' setup parameters are summarized in Table 11. The wireless communication is configured using IEEE 802.11g in ad hoc mode. The simulation uses a Wi-Fi channel with a constant-speed propagation delay model and a log-distance path-loss model. The path-loss exponent is set to 2.5, the reference distance is 1 m, and the reference loss is 40.045 dB, as the default values in ns-3. The transmission power is set to 10 dBm, with 1 dB transmission and reception gains and receiver sensitivity equal to -87 dBm. The rate adaptation is handled using the ideal rate manager, which selects the highest feasible IEEE 802.11g transmission rate according to the channel conditions estimated by the simulator. This setting is used to model adaptive Wi-Fi transmission while avoiding the additional variability introduced by rate-control algorithms, allowing the evaluation to focus on the routing impact of forwarding unavailability prediction. The control and broadcast frames use a robust 6 Mbps OFDM mode. The RTS/CTS threshold is set to 2200 bytes, and the MAC queue size is set to 1000 packets. The simulations are performed over node densities equal to $\{150, 250, 350\}$ vehicles, with a transmission range equal to 230 m. Also, 12 data flows are generated during each run, where every data flow lasts for 100 $sec.$, with a packet size equal to 1 $Kbyte$ and a data transmission rate equal to 1 $Kbyte/sec$. To ensure that the impact of forwarding unavailability is reflected in the routing process, the

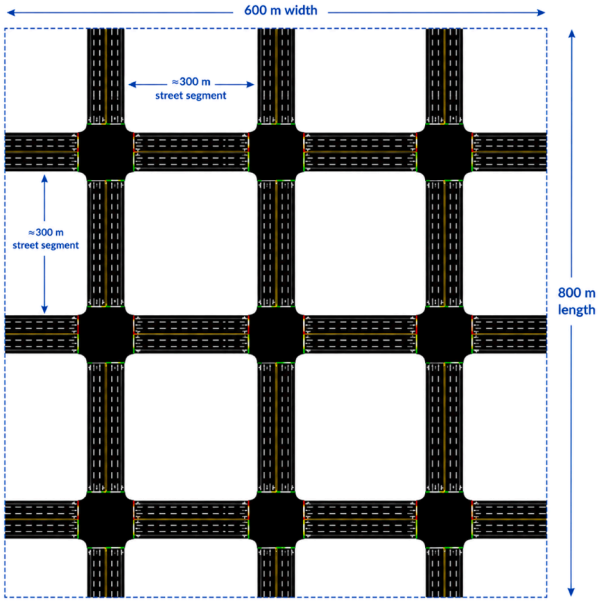


Fig. 3. Simulation area presenting urban IoV environment generated using SUMO.

Table 11
Simulation setup.

Parameter	Value
Network simulator	NS-3
Mobility generator	SUMO
Scenario type	Urban IoV ad hoc network
Road network	9 intersections, 3 lanes per street segment
Simulation area	600×800 m^2
Number of vehicles	$\{150, 250, 350\}$
Wireless standard	IEEE 802.11g
Wireless mode	Ad hoc communication mode
Propagation delay model	Constant-speed propagation delay
Propagation loss model	Log-distance path loss
Propagation loss exponent	2.5
Reference distance	1 m
Reference loss	40.045 dB
Transmission power	10 dBm
Transmission and reception gain	1 dB
Receiver sensitivity	-87 dBm
Transmission Range	230 m
Rate manager	NS3 Ideal WiFi Manager
Control/broadcast mode	6 Mbps OFDM
RTS/CTS threshold	2200 bytes
MAC queue size	1000 packets
Number of data flows	12
Transport protocol	UDP
Packet size	1024 bytes
Application data rate	8192 bps
Transmission period per flow	100 s
Number of iterations	5

senders and receivers are selected at runtime such that they exist around vehicles expected to become unavailable.

The vehicular mobility is generated using SUMO over the urban road network shown in Figure 3. The network represents a rectangular simulation area of 600×800 m^2 and consists of 9 intersections. Each street segment between neighboring intersections is approximately 300 m long, and each road is modeled as a two-way multi-lane street with three lanes per direction. The corresponding vehicle mobility traces are generated using random SUMO trips, where each vehicle is assigned a trip with a start and end location within the simulation area.

During the simulation, each node is randomly assigned one model from the set of 18 trained lightweight XGBoost models. Two distinct simulations are carried out to evaluate the integration of the

prediction model into the routing decisions, namely, weather-context and uncertainty-based simulations. In the weather-context simulations, the routing decisions are evaluated under different weather contexts. The contexts are derived from the dataset binary features and divided into clear sky, rainy, heavy clouds, and light clouds weather. Before the simulation begins, the weather information is retrieved from the dataset with a small random variant on the continuous features that matches the target context. In these simulations, the predicted forwarding unavailability is taken as the ground truth; that is, vehicles predicted to be unavailable are treated as actually unavailable and behave as dropping routers during the run. In the uncertainty-aware simulations, the ground truth is generated from the prediction with uncertainty of 10%, where the predicted availability state is flipped with probability 0.1 before being used as the actual forwarding behavior. Simulations with uncertainty of 10% show the effect of the ML model predictions on the network performance under incorrect unavailability predictions. The NS-3 simulation abstracts the blockchain layer by assuming that the latest forwarding-unavailability prediction is available before route establishment, while smart-contract cost, bytecode size, and latency are evaluated separately in Section 7.3.

The proposed forwarding behavior-aware QoS-OLSR, denoted as OLSR-ML, is compared to the baseline QoS-OLSR [41], denoted as OLSR-LQ. The baseline selects the forwarders based on the link quality and does not consider drops at the forwarding devices due to unavailability. The comparison is performed using the packet delivery ratio (PDR), delay, number of hops, Successful percentage of data flows, and router drop ratio. Each simulation scenario is repeated for 5 iterations, and results are presented as an average.

Under clear-sky weather, presented in Fig. 4, OLSR-ML improves the PDR by 32.15%, 121.54%, and 98.98% for 150, 250, and 350 vehicles, respectively, as shown in Fig. 4a. The successful flows percentage also increases from 70% to 78.33%, from 50% to 85%, and from 46.67% to 81.67%, which corresponds to improvements of 8.33%, 35%, and 35%, respectively, as shown in Fig. 4d. This confirms that the prediction-aware routing is able to complete more data flows by avoiding predicted unavailable relays. However, this improvement is associated with an increase in the average number of hops, as shown in Fig. 4c. The number of hops increases by 26.29%, 63.30%, and 75.43% for 150, 250, and 350 vehicles, respectively. As a result, the delay also increases at higher densities by 149.33% and 124.11% for 250 and 350 vehicles, respectively, as shown in Fig. 4b. At 150 vehicles, the delay slightly decreases by 3.67%, which indicates that when the network is less dense, OLSR-ML can avoid unavailable relays without introducing additional route delay.

Under rainy and heavy cloud weather, as presented in Fig. 5, OLSR-ML also improves the PDR compared to OLSR-LQ. As shown in Fig. 5a, the PDR increases by 37.12%, 140.43%, and 108.17% for 150, 250, and 350 vehicles, respectively. The successful flows percentage, shown in Fig. 5d, also increases by 18.33, 13.33, and 31.67 percentage for 150, 250, and 350 vehicles, respectively. This indicates that the proposed routing protocol maintains better packet delivery even under weather conditions that are associated with higher forwarding unavailability. The number of hops increases by 32.46%, 36.66%, and 43.35%, respectively, as shown in Fig. 5c. The delay increases by 12.38% and 117.80% for 150 and 350 vehicles, respectively, while it slightly decreases by 1.92% for 250 vehicles, as shown in Fig. 5b. Therefore, the delay increase is not consistent across all densities, but mainly appears when OLSR-ML selects longer alternative paths to avoid unavailable vehicles.

Under light cloud weather, presented in Fig. 6, OLSR-ML improves the PDR by 22.62%, 61.89%, and 50.46% for 150, 250, and 350 vehicles, respectively, as shown in Fig. 6a. The successful flows percentage also improves from 73.33% to 85%, from 60% to 86.67%, and from 40% to 66.67%, which corresponds to improvements of 11.67, 26.67, and 26.67 percentage points, respectively, as shown in Fig. 6d. The number of hops increases by 4.34%, 22.65%, and 63.26%, respectively, as shown in Fig. 6c. Similarly, the delay slightly decreases by 1.08% for 150 vehicles, while it increases by 7.05% and 52.67% for 250 and 350 vehicles, respectively, as shown in Fig. 6b. This shows that the cost of avoiding unavailable vehicles becomes more visible at higher densities, where the protocol needs to select longer paths to maintain stable forwarding.

Fig. 7 presents the network performance when the forwarding unavailability prediction is affected by 10% uncertainty. In this scenario, the predicted forwarding unavailability state is flipped with probability 0.1 before being used as the actual forwarding behavior. Therefore, this experiment evaluates whether OLSR-ML can still improve the routing performance when the prediction is not fully accurate. Since the Uber Rides Peru labels are used as a proxy for forwarding availability rather than direct packet-forwarding observations, the uncertainty-aware simulation is also conducted by perturbing the predicted availability state before using it as the actual forwarding behavior.

The results show that OLSR-ML still outperforms OLSR-LQ in terms of PDR under all node densities. The PDR improves by 16.11%, 37.97%, and 49.27% for 150, 250, and 350 vehicles, respectively. This confirms that even under prediction uncertainty, the forwarding behavior-aware routing protocol can still avoid a considerable number of unavailable vehicles and deliver more packets than the baseline routing protocol. The successful flows percentage also improves from 76.67% to 78.33%, from 60% to 83.33%, and from 50% to 78.33% for 150, 250, and 350 vehicles, respectively. These results correspond to improvements of 1.67, 23.33, and 28.33 percentage points, respectively. Hence, the benefit of OLSR-ML becomes more visible as the network density increases, where the baseline has more chance to select vehicles that later become unavailable.

The router drop percentage also decreases significantly when OLSR-ML is used. The drop percentage decreases from 18.53% to 1.55% for 150 vehicles, from 28.53% to 9.63% for 250 vehicles, and from 28.68% to 10.20% for 350 vehicles. This corresponds to reductions of 91.64%, 66.24%, and 64.44%, respectively. This result is important because it shows that the proposed protocol not only improves the final packet delivery, but also reduces the dependence on routers that drop packets during forwarding. Therefore, the prediction-aware routing decision remains effective even when part of the prediction output is incorrect.

These reliability improvements should be interpreted together with the additional routing cost introduced by proactive avoidance. In OLSR-ML, vehicles predicted to become unavailable are assigned lower willingness and QoS roles, while the routing cost of links connected to them is increased. As a result, the protocol avoids unreliable relays and selects alternative routes that are more likely to remain available. However, these alternative routes may be longer than the routes selected by OLSR-LQ. This explains the increase in the number of hops under OLSR-ML by 14.50%, 24.51%, and 45.40% for 150, 250, and 350 vehicles, respectively. Therefore, the improvement in PDR, successful flows, and router-drop reduction comes at the cost of higher routing path length in some cases.

The delay results should also be interpreted as part of this reliability-overhead trade-off. The delay increases by 7.98% and 25.00% for 150 and 250 vehicles, respectively, while it decreases by 28.20% for 350 vehicles. The increase in delay for 150 and 250 vehicles is justified by the higher number of successful flows and longer paths selected by OLSR-ML. In the baseline OLSR-LQ, some flows are dropped earlier due to unavailable routers, which can reduce the measured average delay because fewer packets are successfully delivered over difficult paths. In contrast, OLSR-ML delivers more flows successfully, including flows that require longer alternative routes, which increases the average delay of delivered packets. At 350 vehicles, OLSR-ML improves the PDR and successful flows while also reducing the delay, which indicates that avoiding unavailable routers can also prevent packets from experiencing delay caused by unstable forwarding paths. Accordingly, the proposed protocol improves routing reliability, but this improvement may require accepting longer routes and, in some scenarios, higher delay.

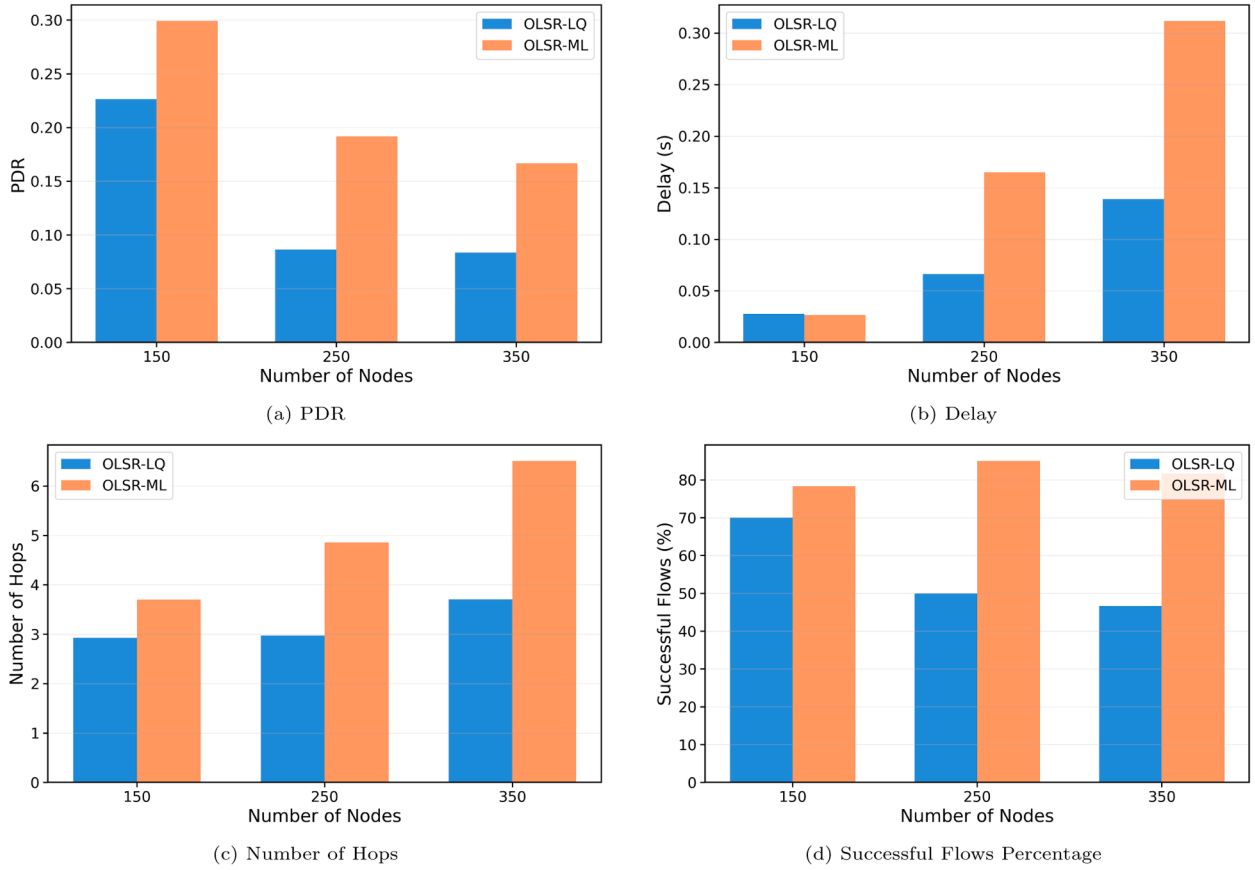


Fig. 4. Network performance during clear-sky weather.

7.3. Smart contracts cost and security analysis

In this section, the deployment cost and the cost of interacting with the implemented smart contracts are presented and discussed. The Ether values reported in Tables 12 and 13 are computed using the standard gas price of 0.567 *Gwei* as of Apr. 27th 2026.

For the Prediction Registry contract, the deployment cost is 1,721,734 gas, which corresponds to 0.00098 Ether. This cost is paid only once when the contract is deployed. However, the interaction cost of the functions is relatively low compared to the deployment. The lowest interaction cost is 31,153 gas, while *setNodePredictor()* costs 48,253 gas and *addPredictionFromContract()* costs 99,818 gas. The highest interaction cost is observed for *upsertMyConfig()* with 430,051 gas, which is expected since this function stores and updates the selected features, scaling factor, and normalization parameters of the vehicle. Hence, the cost of *upsertMyConfig()* is higher than the cost of uploading a prediction, because it writes larger arrays and multiple parameters to storage. In comparison, *uploadPrediction()* costs 90,633 gas, which remains relatively low since it adds only the prediction value and timestamp to the history buffer.

The computational feasibility of the Forwarding Behavior Predictor is evaluated by varying the number of trees in the lightweight XGBoost model and measuring the deployment gas, runtime gas, and deployed bytecode size. As presented in Table 13, increasing the number of trees from 10 to 20 increases the deployment gas from 2,535,716 to 3,548,656 gas and increases the deployed bytecode size from 11.15 KB to 15.72 KB. This is expected because additional trees generate more contract logic. In contrast, the runtime cost of *predictAndUpload()* increases only slightly, from 120,249 gas for 10 trees to 125,339 gas for 20 trees. This shows that the model size mainly affects deployment overhead and bytecode size, while the runtime prediction overhead remains limited for the evaluated lightweight models.

The adopted 10-tree model is therefore selected as the on-chain predictor because it provides the lowest deployment gas and smallest bytecode size among the evaluated configurations, while maintaining strong prediction performance as reported in Table 7. Larger models, including the 300-tree reference XGBoost model, are used only as off-chain

Table 12
Prediction Registry implementation cost.

Contract	Function	Gas	Ether
Prediction Registry	Deployment	1,721,734	0.00098
	setNodePredictor()	48,253	0.000027
	upsertMyConfig()	430,051	0.00024
	addPredictionFromContract()	99,818	0.000056
	setNodePredictor()	31,153	0.000018
	uploadPrediction()	90,633	0.000051

Table 13
Cost and bytecode size of the Forwarding Behavior Predictor with different numbers of trees.

Number of Trees	Deployment		predictAndUpload()		Bytecode Size
	Gas	Ether	Gas	Ether	
10	2,535,716	0.00144	120,249	0.000068	11.15 KB
15	3,078,859	0.00175	122,801	0.000069	13.60 KB
20	3,548,656	0.00201	125,339	0.000071	15.72 KB

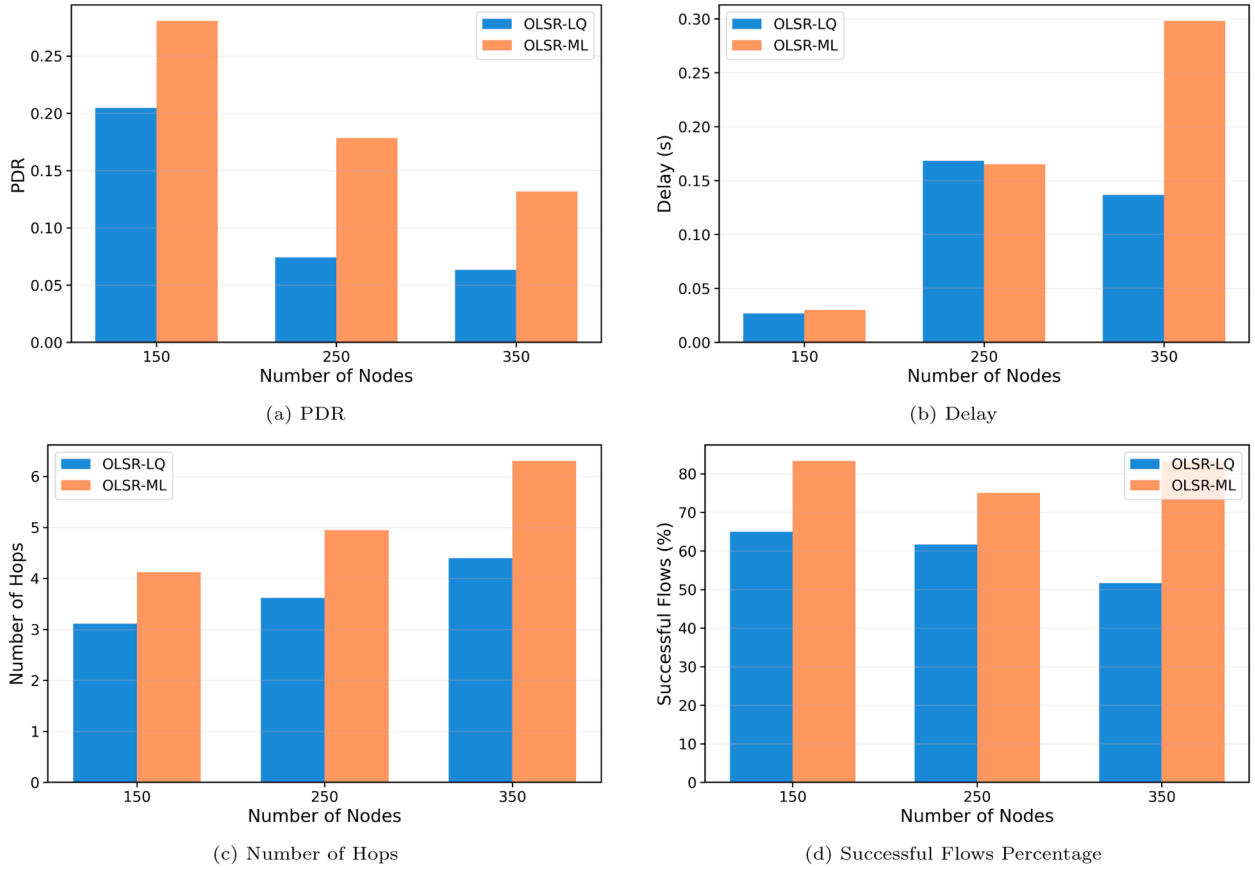


Fig. 5. Network performance during rainy and heavy cloud weather.

upper-bound baselines and are not intended for direct smart-contract deployment, since they would substantially increase generated contract code size and deployment overhead.

Although the reported gas costs quantify the computational effort required to deploy and execute the smart contracts, the operational feasibility of the proposed framework also depends on the blockchain deployment model. In this work, the blockchain layer is assumed to operate as a permissioned consortium blockchain rather than an open public blockchain. In the considered IoV environment, the consortium blockchain can be maintained by authorized entities, such as RSUs, edge servers, road operators, service providers, or trusted network stakeholders. Under this deployment model, transaction confirmation latency depends on the selected consensus mechanism, validator configuration, block interval, and network conditions.

It is important to distinguish between local smart-contract execution time and end-to-end blockchain confirmation latency. To evaluate the computational overhead of the implemented contract logic, the *predictAndUpload()* transaction was executed on a local single-node Ganache instance. The measured local execution and mining time was approximately 1.20 ms, indicating that the lightweight XGBoost inference, which mainly consists of tree traversal, scaled integer operations, and sigmoid approximation, introduces negligible local EVM execution overhead. However, this value does not represent the confirmation latency of a real multi-validator consortium blockchain. In practical consortium deployments, the confirmation delay is dominated by consensus execution, block generation, validator communication, and transaction finality. Existing private-blockchain benchmarking of Hyperledger Besu shows that, under a 1 s block-period configuration, write transactions can achieve average latency below 1.5 s, while increasing the block period from 1 s to 5 s significantly increases latency because transactions wait longer for block proposal [42]. Therefore, for a properly configured consortium blockchain, the expected confirmation latency for prediction transactions is on the order of low seconds, rather than milliseconds. Since the proposed prediction is performed once before trip start or route establishment, this low-second confirmation delay is acceptable for pre-routing prediction dissemination.

Regarding memory and storage requirements, the compiled runtime bytecode size of the lightweight 10-tree Forwarding Behavior Predictor is 11,419 bytes, or approximately 11.15 KB. The Prediction Registry runtime bytecode size is 7443 bytes, or approximately 7.27 KB. These values represent the deployed contract logic and remain fixed after deployment for a given predictor and registry implementation. The dynamic on-chain storage is mainly associated with the registry records, including the vehicle configuration, selected feature indices, normalization parameters, and bounded prediction history. Therefore, storage overhead grows with the number of registered vehicles and the number of retained prediction records. Since the prediction history is maintained using a bounded buffer, the storage requirement does not grow indefinitely over time for each vehicle. In addition, since each vehicle-specific predictor is executed independently, the inference complexity of one prediction does not increase with the total number of vehicles in the IoV network. However, deploying one predictor per vehicle means that the cumulative deployment and storage overhead increases approximately linearly with the number of participating vehicles.

The current framework assumes that the forwarding availability prediction remains valid during the corresponding trip. Therefore, prediction updates are performed once before trip start or route establishment, rather than continuously during packet forwarding. This design reduces blockchain interaction overhead and avoids frequent routing-table changes caused by repeated prediction updates. However, in highly dynamic scenarios, mid-trip changes in route, weather, or user behavior may require adaptive prediction updates. Frequent updates

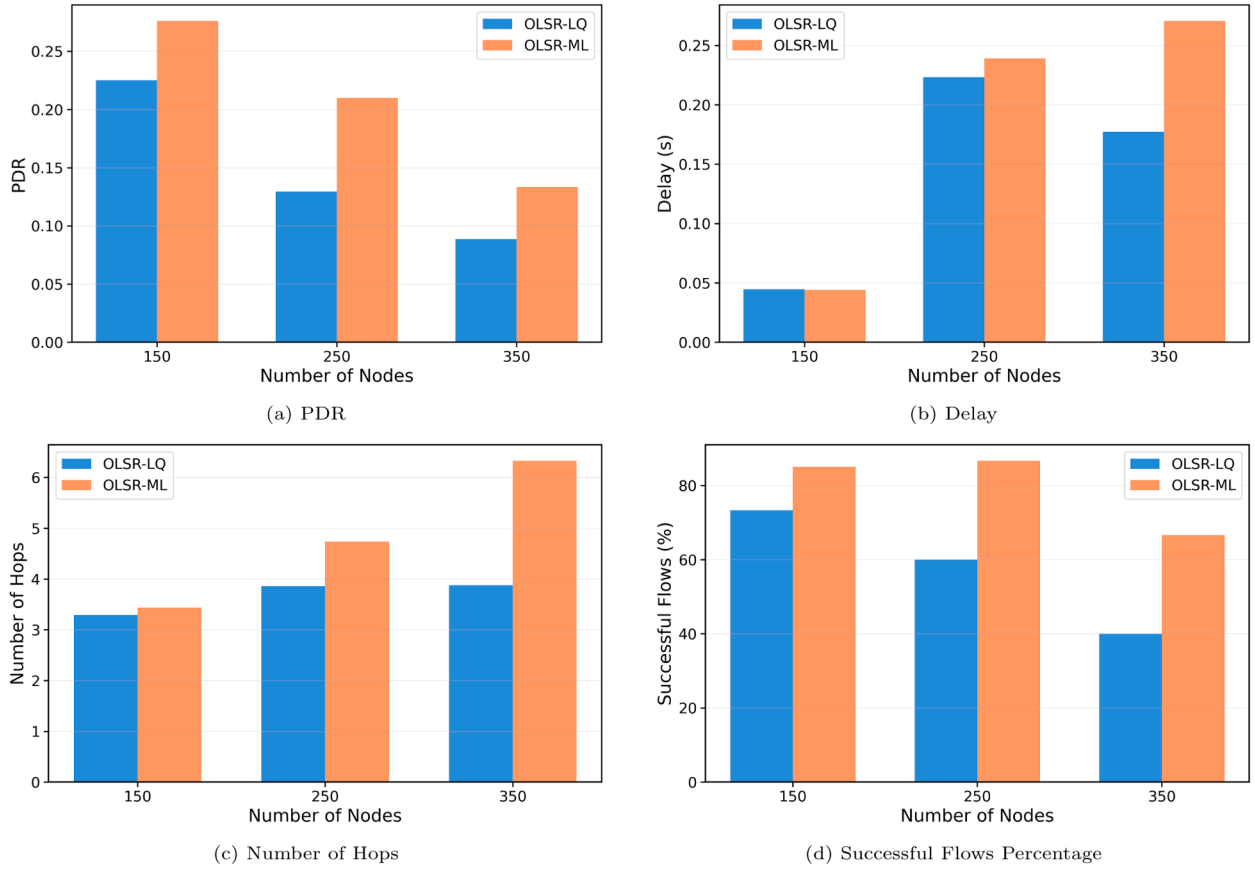


Fig. 6. Network performance during light cloud weather.

would increase blockchain transactions and may introduce additional routing instability if link costs are repeatedly changed. Designing adaptive update policies that balance prediction freshness, blockchain overhead, and routing stability is left as future work.

Although the blockchain layer provides transparency, immutability, and public verifiability of the recorded prediction outputs, it does not eliminate all security and privacy risks. Therefore, the main threats that may affect the proposed framework, including false prediction reporting, compromised vehicle-specific models, malicious smart contract interactions, and privacy leakage from contextual trip information are discussed below.

First, false prediction reporting may occur if a vehicle attempts to announce a fabricated forwarding availability probability to influence routing decisions. In the proposed framework, this risk is reduced by requiring predictions to be uploaded through the corresponding Forwarding Behavior Predictor smart contract rather than being directly self-reported as arbitrary values. The Prediction Registry stores the address of the authorized predictor contract associated with each vehicle, and prediction updates are accepted only from the registered predictor contract. Moreover, the prediction parameters, such as the selected feature indices, scaling factor, and normalization parameters, are recorded in the registry to support validation. Therefore, other vehicles can re-execute the same on-chain prediction function using the announced input features and compare the generated probability with the recorded value. This makes fabricated prediction reporting detectable, assuming that the registered predictor contract and input features are valid.

Second, a vehicle may attempt to deploy or register a compromised prediction model that intentionally produces misleading availability probabilities. Blockchain immutability ensures that the deployed model logic cannot be silently modified after deployment, but it does not by itself guarantee that the deployed model is honest or well-trained. Therefore, model registration should be supported by validation mechanisms, such as checking the model metadata, verifying the training procedure, auditing the generated smart contract code, or approving predictor contracts through a consortium of validators, RSUs, or network operators. In the current framework, the focus is on making prediction execution transparent and reproducible once the predictor is registered. The validation of the training process and the prevention of malicious model registration are considered important extensions for future work.

Third, malicious smart contract interactions may target the Prediction Registry by attempting unauthorized prediction updates, replaying outdated predictions, or modifying another vehicle's configuration. This is mitigated through access-control logic in the smart contracts. Prediction uploads are restricted to registered Forwarding Behavior Predictor contracts, and user configuration updates are associated with the corresponding vehicle identity. In addition, each prediction is stored with a timestamp, allowing routing nodes to check prediction freshness and ignore outdated entries. Nevertheless, additional protection mechanisms, such as stricter identity management, rate limiting, validator approval, and event monitoring, may be required in large-scale deployments to reduce spam transactions or repeated unnecessary updates.

Fourth, privacy leakage may arise because contextual trip information, such as trip start and end locations, temporal features, and weather-related inputs, can reveal sensitive mobility patterns if stored or shared publicly. In the proposed framework, this information is used to support prediction validation, but exposing raw trip context on-chain may create privacy concerns. Therefore, practical deployments should minimize the amount of contextual data stored directly on-chain. Possible mitigation strategies include storing only scaled or discretized feature values, sharing trip context through off-chain channels, recording cryptographic hashes or commitments of input features, or using zero-knowledge proofs to prove that a prediction was computed correctly

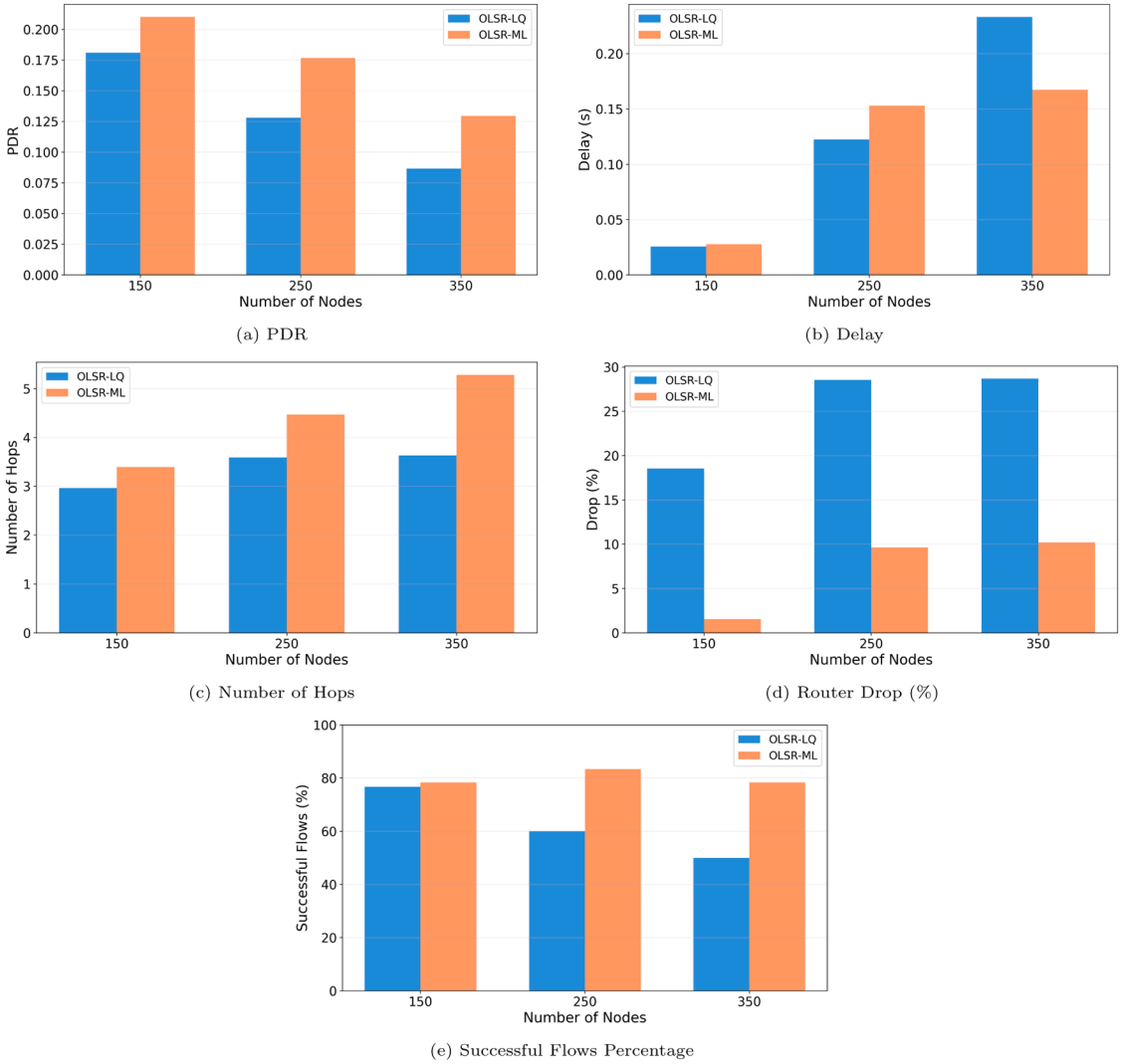


Fig. 7. Network performance comparison with 10% uncertainty in ML model prediction.

without revealing all raw trip attributes. Another option is to limit the spatial precision of start and end locations so that prediction validation remains possible without exposing exact mobility traces.

7.4. Dataset scale, behavioral generalization, and future directions

The forwarding behavior prediction models in this work are trained and evaluated using the Uber Rides Peru dataset, which includes 18 drivers and 10,551 trip records. While this dataset provides the temporal, spatial, and weather-related contextual features required to model forwarding availability as a rational and trip-dependent behavior, its limited scale restricts the diversity of driving patterns, trip contexts, and behavioral profiles captured during training. Since the proposed framework trains a dedicated lightweight XGBoost model per vehicle, the architecture is not based on a single population-level model that must represent all IoV participants. Instead, each vehicle model reflects the historical behavior of its corresponding vehicle. In a practical deployment, each participating vehicle would collect its own contextual forwarding-service history and train or update its own predictor.

However, this vehicle-specific design does not eliminate generalization concerns when the framework is applied to larger and more heterogeneous IoV networks. Each vehicle model may be trained on a limited number of historical trips, and therefore the learned patterns may not generalize well to trip conditions, routes, or contextual combinations that differ substantially from those observed during training. This issue is more significant for newly joining vehicles with little or no historical data, which creates a cold-start problem. In addition, larger IoV networks may include more diverse behavioral profiles, such as commuters, occasional drivers, and full-time drivers, which may require periodic retraining or model updating to keep each predictor aligned with the vehicle's evolving behavior.

The larger network simulations with 150, 250, and 350 vehicles evaluate the scalability of the routing integration mechanism by applying the trained predictors across different network densities. Therefore, the simulation results demonstrate the network-level effect of integrating forwarding behavior prediction into QoS-OLSR, while the behavioral generalization of the prediction models remains limited by the available dataset. Future work should validate the proposed framework using larger and more heterogeneous IoV-specific datasets, including more vehicles, diverse mobility patterns, incentive conditions, and explicit forwarding availability labels.

The current framework has several limitations that should be addressed in future work. First, the prediction model should be validated using larger IoV-specific datasets with explicit forwarding availability labels, instead of relying on proxy service-completion labels. Such datasets should include richer contextual and resource-related factors, such as packet-forwarding behavior, bandwidth usage, battery level, incentive conditions, and user preferences. Second, the current

model predicts whether a vehicle may become unavailable during a trip, but it does not estimate the exact time or duration of unavailability. Future work should develop time-aware prediction models so that routing protocols can avoid vehicles only during their predicted unavailable intervals. Third, the framework assumes that the pre-trip prediction remains valid during the trip; therefore, adaptive mid-trip prediction updates should be investigated while balancing prediction freshness, blockchain overhead, and routing stability. Fourth, the vehicle-specific predictors may face cold-start and generalization challenges when vehicles have limited historical data, which motivates future work on transfer learning, federated learning, or clustering-based model sharing. Finally, large-scale deployment requires more scalable and privacy-preserving blockchain designs, such as shared predictor templates, off-chain model storage with on-chain hashes, and cryptographic mechanisms that validate predictions without exposing sensitive trip information.

8. Conclusion

This paper proposes a blockchain-assisted context-aware forwarding unavailability prediction framework for crowdsourced IoV. The work addressed the problem that vehicles in crowdsourced IoV may become unavailable for forwarding during their trips due to contextual reasons, even when they are not malicious. To tackle this problem, forwarding participation was modeled as a context-dependent behavior of rational vehicles, and the forwarding unavailability of each vehicle was predicted before route establishment using lightweight XGBoost models trained on temporal, spatial, and weather-related features. The trained models were then transformed into executable smart contracts and deployed on-chain through Forwarding Behavior Predictor contracts, while the generated predictions and the corresponding validation-related parameters were recorded and disseminated through a Prediction Registry smart contract. The predicted forwarding unavailability was integrated into QoS-OLSR to proactively avoid vehicles expected to become unavailable by reducing their role in relay selection and penalizing the routing cost of connected links. The results showed that the lightweight XGBoost model provides a suitable tradeoff between prediction performance and on-chain implementation cost, which makes it practical for blockchain-based execution. The simulation results also showed that the forwarding behavior-aware QoS-OLSR improves the packet delivery ratio compared to the baseline QoS-OLSR under different weather contexts and under prediction uncertainty. This improvement was achieved at the cost of longer routing paths and higher delay in most cases, which is expected since the protocol selects more stable forwarding vehicles instead of shorter routes through vehicles likely to become unavailable. In addition, the gas cost analysis showed that the lightweight predictor and registry smart contracts remain feasible for on-chain deployment and interaction. Future work will address the main limitations of the current framework by validating it on larger IoV-specific datasets with explicit forwarding availability labels, developing time-aware models that predict both the start time and duration of unavailability, and designing adaptive mid-trip update policies that balance prediction freshness, blockchain overhead, and routing stability. Additional research is also needed to improve scalability through shared or parameterized predictor contracts, support cold-start vehicles with limited historical data, and protect trip-context privacy during prediction validation. These directions can further improve the practicality, fairness, and reliability of forwarding behavior-aware routing in large-scale crowdsourced IoV networks.

CRediT authorship contribution statement

Huda Abualola: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Conceptualization; **Shakti Singh:** Writing – review & editing, Supervision, Conceptualization; **Rabeb Mizouni:** Writing – review & editing, Supervision, Conceptualization; **Hadi Otrok:** Writing – review & editing, Supervision, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This research was funded by Khalifa University of Science and Technology through TII under Project ID: KU-EXT-TII-2021-8434000405.

Data availability

Data will be made available on request.

References

[1] A. Kumar, L. Vishwakarma, D. Das, R-Pbft: a secure and intelligent consensus algorithm for internet of vehicles, Veh. Commun. 41 (2023) 100609. https://doi.org/10.1016/j.vehcom.2023.100609
[2] R. Pirmagomedov, A. Ometov, D. Moltchanov, X. Lu, R. Kovalchukov, E. Olshannikova, S. Andreev, Y. Koucheryavy, M. Dohler, Applying blockchain technology for user incentivization in mmwave-based mesh networks, 8, IEEE Access, 2020. https://doi.org/10.1109/ACCESS.2020.2979135
[3] H. Abualola, R. Mizouni, S. Singh, H. Otrok, Meshchain: a comprehensive blockchain-based framework for mesh networks, Ad Hoc Netw. 175 (2025) 103860. https://doi.org/10.1016/j.adhoc.2025.103860
[4] M. Kadadha, S. Singh, R. Mizouni, H. Otrok, A context-Aware blockchain-Based crowdsourcing framework: open challenges and opportunities, IEEE Access 10 (2022) 93659–93673. https://doi.org/10.1109/ACCESS.2022.3203850
[5] Z. Husain, J. Dutta, S. Singh, R. Mizouni, T.H. El-Fouly, H. Otrok, Context-aware energy auctions on blockchain for mobile ev charging, Sustain. Cities Soc. 130 (2025) 106548. https://doi.org/10.1016/j.scs.2025.106548
[6] A. Mehmood, M. Afaq, F. Mehmood, W.-C. Song, Edge-based proactive and stable two-tier routing for iov, Future Gener. Comput. Syst. 179 (2026) 108367. https://doi.org/10.1016/j.future.2026.108367
[7] M. Abououf, S. Singh, H. Otrok, R. Mizouni, E. Damiani, Machine learning in mobile crowd sourcing: a behavior-based recruitment model, 22 1 (2021). https://doi.org/10.1145/3451163
[8] N. Ilakkiya, A. Rajaram, A secured trusted routing using the structure of a novel directed acyclic graph-blockchain in mobile ad hoc network internet of things environment, Multimed. Tools Appl.. http://dx.doi.org/10.1007/s11042-024-18845-1. https://doi.org/10.1007/s11042-024-18845-1
[9] Y. Inedjaren, M. Maachaoui, B. Zeddini, J.-P. Barbot, Blockchain-based distributed management system for trust in vanet, Veh. Commun. 30 (2021) 100350. https://doi.org/10.1016/j.vehcom.2021.100350
[10] H. Xia, Z. Jia, X. Li, L. Ju, E.H.-M. Sha, Trust prediction and trust-based source routing in mobile ad hoc networks, 11, (7) Elsevier, 2013. https://doi.org/10.1016/j.adhoc.2012.02.009
[11] M. Bampatsikos, I. Politis, T. Ioannidis, C. Xenakis, Trust score prediction and management in iot ecosystems using markov chains and madm techniques, IEEE Trans. Consum. Electron. 71 (1) (2025) 862–882. https://doi.org/10.1109/TCE.2025.3531045
[12] A.A. Mahamune, M. Chandane, Trust-based co-operative routing for secure communication in mobile ad hoc networks, Digit. Commun. Netw. 10 (4) (2024) 1079–1087. https://doi.org/10.1016/j.dcan.2023.01.005
[13] S.M. Hassan, M.M. Mohamad, F.B. Muchtar, Enhancing manet security through long short-term memory-based trust prediction in location-aided routing protocols, IEEE Access 13 (2025) 120142–120168. https://doi.org/10.1109/ACCESS.2025.3572619
[14] H. Abualola, H. Otrok, Stable coalitions for urban-vanet: a hedonic game approach, Veh. Commun. 30 (2021) 100355. https://doi.org/10.1016/j.vehcom.2021.100355
[15] P. Priyadarshini, A. Kumar, Rnn-gan probabilistic forecasting for chance-constrained UAV-VANET routing, Ad Hoc Netw. 188 (2026) 104243. https://doi.org/10.1016/j.adhoc.2026.104243
[16] J. Liu, L. Yang, N. Kumar, A.M. Almuhaideb, K.I. Kostromitin, P. Zhang, Trajectory prediction training scheme in vehicular ad-hoc networks based on federated learning, Ad Hoc Netw. 178 (2025) 103917. https://doi.org/10.1016/j.adhoc.2025.103917
[17] A. Alharthi, Q. Ni, R. Jiang, M.A. Khan, A computational model for reputation and ensemble-based learning model for prediction of trustworthiness in vehicular ad hoc network, IEEE Internet Things J. 10 (20) (2023) 18248–18258. https://doi.org/10.1109/JIOT.2023.3279950
[18] A. Alagha, M. Kadadha, R. Mizouni, S. Singh, J. Bentahar, H. Otrok, Uav-assisted internet of vehicles: a framework empowered by reinforcement learning and blockchain, Veh. Commun. (2025). https://doi.org/10.1016/j.vehcom.2025.100874

[19] B. Zhang, X. Wang, R. Xie, C. Li, H. Zhang, F. Jiang, A reputation mechanism based deep reinforcement learning and blockchain to suppress selfish node attack motivation in vehicular ad-hoc network, Future Gener. Comput. Syst. 139 (2023) 17–28. https://doi.org/10.1016/j.future.2022.09.010

[20] Y. Li, Y. Cao, Y. Zhuang, J. Li, G. Du, J. Chen, Blockchain-enabled trust management with location privacy preservation in vehicular ad hoc networks, IEEE Internet Things J. 11 (14) (2024) 24659–24671. https://doi.org/10.1109/JIOT.2024.3350694

[21] D.S. Bhatti, S. Saleem, A. Imran, H.J. Kim, K.-I. Kim, K.-C. Lee, Detection and isolation of wormhole nodes in wireless ad hoc networks based on post-wormhole actions, Sci. Rep. 14 (1) (2024) 3428.

[22] J. Ryu, S. Kim, Trust system- and multiple verification technique-based method for detecting wormhole attacks in manets, IEEE Access 12 (2024) 16266–16275. https://doi.org/10.1109/ACCESS.2024.3355467

[23] E. Kristianto, P.-C. Lin, R.-H. Hwang, Misbehavior detection system with semi-supervised federated learning, Veh. Commun. 41 (2023) 100597. https://doi.org/10.1016/j.vehcom.2023.100597

[24] M.A. Abdelmaguid, H.S. Hassanein, M. Zulkernine, Samm: situation awareness with machine learning for misbehavior detection in vanet, in: Proceedings of the 17Th International Conference on Availability, Reliability and Security, ARES '22, Association for Computing Machinery, New York, NY, USA, 2022. https://doi.org/10.1145/3538969.3543788

[25] S. Sharma, R. Thakur, A trust-centric contract incentive mechanism for collaborative parked computing vehicle framework, Veh. Commun. 59 (2026) 101012. https://doi.org/10.1016/j.vehcom.2026.101012

[26] F. Mansouri, M. Tarhouni, B. Alaya, S. Zidi, A distributed intrusion detection framework for vehicular ad hoc networks via federated learning and blockchain, Ad Hoc Netw. 167 (2025) 103677. https://doi.org/10.1016/j.adhoc.2024.103677

[27] R. Singh, G.P. Dubey, Secure and congestion-aware optical switching framework for efficient routing in flying ad-hoc networks, Opt. Fiber Technol. 99 (2026) 104582. https://doi.org/10.1016/j.yofte.2026.104582

[28] T. Jaya, N. Maradona, Multipath routing and intrusion detection in mobile ad hoc network using hybrid optimization-based deep maxout network, Wirel. Netw. 32 (2) (2026) 1033–1055. https://doi.org/10.1007/s11276-026-04094-7

[29] M. Deivakani, M.S. Sheela, K. Priyadarsini, Y. Farhaoui, An intelligent security mechanism in mobile ad-hoc networks using precision probability genetic algorithms (ppga) and deep learning technique (stacked lstm), Sustain. Comput.: Inform. Syst. 43 (2024) 101021. https://doi.org/10.1016/j.suscom.2024.101021

[30] Z.A. Haider, M. Fayaz, I. Ullah, M. Altayeb, F. Alblehai, A. Tolba, D2vauth: Efficient drone-to-vehicle authentication using blockchain and hyperelliptic curve cryptography, IEEE Trans. Consum. Electron. 72 (2) (2026) 2775–2786. https://doi.org/10.1109/TCE.2026.3686430

[31] Z. Husain, R. Mizouni, T.H.M. El-Fouly, S. Singh, H. Otrok, Groupcharge: a blockchain-based coalition and auction framework for mobile ev charging, IEEE Trans. Intell. Transp. Syst. 27 (3) (2026) 3493–3504. https://doi.org/10.1109/TITS.2025.3644031

[32] I. Ullah, Z.A. Haider, X. Su, C. Choi, A. Arishi, H. Bilal, Vehicular communications for autonomous and intelligent transportation systems: a comprehensive survey, Digit. Commun. Netw.. https://doi.org/10.1016/j.dcan.2026.06.003

[33] H. Lin, J. Zhao, X. Zhu, L. Wang, Trust evaluation mechanism for the internet of vehicles based on filtering algorithm, Comput. Netw. 271 (2025) 111583. https://doi.org/10.1016/j.comnet.2025.111583

[34] R. Nasser, A. Alagha, S. Singh, R. Mizouni, H. Otrok, J. Bentahar, Poisoning behavioral-based worker selection in mobile crowdsensing using generative adversarial networks, J. Netw. Comput. Appl. 242 (2025) 104236. https://doi.org/10.1016/j.jnca.2025.104236

[35] I. Ullah, I. Khalil, X. Bai, S. Garg, G. Kaddoum, M.S. Hossain, An ensemble-based hybrid model for the detection of attacks in the internet of vehicular things, IEEE Trans. Intell. Transp. Syst. 26 (10) (2025) 17914–17927. https://doi.org/10.1109/TITS.2025.3547999

[36] Z.A. Haider, I. Ullah, A. Abdusalomov, M. Shah, M.Z. Khan, B.A. Zneid, Edge-intelligent semantic aggregation in blockchain-secured 6g uav-assisted internet of vehicles, J. Electron. Sci. Technol. 24 (1) (2026) 100350. https://doi.org/10.1016/j.jnlest.2026.100350

[37] T. Guo, D. Xu, X. Zhu, Mmjp: a mobility and congestion-Aware routing protocol for flying ad hoc networks, IEEE Trans. Veh. Technol. (2026) 1–16. https://doi.org/10.1109/TVT.2026.3679327

[38] X. Li, H. Sun, Prediction-based reactive-greedy routing protocol for flying ad hoc networks, Wirel. Netw. 31 (3) (2025) 2893–2907. https://doi.org/10.1007/s11276-025-03925-3

[39] I. Ullah, S.K. Singh, D. Adhikari, H. Khan, W. Jiang, X. Bai, Multi-agent reinforcement learning for task allocation in the internet of vehicles: exploring benefits and paving the future, Swarm Evol. Comput. 94 (2025) 101878. https://doi.org/10.1016/j.swevo.2025.101878

[40] N. Elsokkary, Uber rides peru dataset, 2023, (GitHub repository). https://github.com/NadaElsokkary/UberRidesPeru.

[41] H. Abualola, M. Kadadha, R. Mizouni, S. Singh, H. Otrok, M. Baddeley, F. Betene, J.-P. Giacalone, Qos-olsr 2.0: a quality-of-service optimized link state routing protocol for mesh networks, in: 2023 International Wireless Communications and Mobile Computing (IWCMC), 2023, pp. 1040–1045. https://doi.org/10.1109/IWCMC58020.2023.10182849

[42] C. Fan, C. Lin, H. Khazaei, P. Musilek, Performance analysis of hyperledger besu in private blockchain, in: 2022 IEEE International Conference on Decentralized Applications and Infrastructures (DAPPS), 2022, pp. 64–73. https://doi.org/10.1109/DAPPS55202.2022.00016